PHENIKAA UNIVERSITY

FACULTY OF ELECTRICAL AND ELECTRONIC ENGINEERING

ADVANCED DEEP LEARNING COURSE

REAL-TIME PLANT DISEASE DIAGNOSIS

STUDENTS:

22010846 – LE THI THUY

22011077 – HOANG THI THAO

22011108 – CHU THI CHUC LINH

Advisor: Ph.D Le Minh Huy

HA NOI, October 2026

ADVANCED DEEP LEARNING COURSE REPORT

Le Thi Thuy – 22010846

Hoang Thi Thao – 22011077

Chu Thi Chuc Linh – 22011108

Advisor: **Ph.D: Le Minh Huy**

Abstract

Plant diseases significantly affect agricultural yield and quality. Manual diagnosis relies on professional expertise, is time-consuming, and can be challenging with field images due to complex backgrounds, varying lighting conditions, and data imbalance. Therefore, deep learning offers a promising approach for automated leaf disease detection and classification.

This study develops a deep learning-based leaf image classification system using the PlantDoc dataset. After data cleaning, the "Tomato two-spotted spider mites leaf" class was removed due to insufficient samples, resulting in 27 classes. Images were resized to 160 × 160 × 3, and data augmentation was applied to improve generalization.

Three neural network architectures – ResNet18, SE-ResNet18, and DSCNet—were trained from scratch under the same experimental setup. DSCNet combines Depthwise Separable Convolution with residual connections to reduce parameters and computational cost. The models were evaluated using validation macro-F1, test accuracy, test Top-3 accuracy, and test macro-F1 across multiple random seeds to assess result stability.

Multi-seed results show that DSCNet achieves the best performance among the three models trained from scratch, with a test accuracy of 37.3% ± 2.1%, Top-3 accuracy of 62.1% ± 1.1%, and macro-F1 of 0.354 ± 0.016, while using only approximately 2.70 million parameters. An ablation study, Grad-CAM analysis, and occlusion sensitivity analysis were also conducted to examine training components and the model's focus on disease-related regions.

The path to the drive folder containing the code and data: https://drive.google.com/drive/folders/16UoEsr0blYa5muAicUFXeSTntXHzDm5Q?usp=sharing

Link to the presentation video: https://www.youtube.com/watch?v=gMVuPCDA95U

TABLE OF CONTENTS

LIST OF FIGURES **5**

LIST OF TABLES **7**

CHAPTER 1: PROBLEM SITUATION AND PROBLEM DEFINITION **8**

1.1. Problem situation 8

1.2. Problem definition 9

CHAPTER 2: OBJECTIVES AND DATA PROCESSING **11**

2.1. General objectives 11

2.2. Research objectives 11

2.3. Data processing 12

2.3.1. Data cleaning **12**

2.3.2. Data split **13**

2.3.3. Image preprocessing **14**

2.3.4. Strong augmentation **15**

CHAPTER 3: ARCHITECTURAL MODELS AND METHODS **16**

3.1. Model ResNet18 16

3.2. Model SE-ResNet18 17

3.3. Model DSCNet 18

3.4. Explanatory analysis using Grad-CAM 20

CHAPTER 4: RESULTS AND EVALUATION **26**

4.1. Evaluation protocol 26

4.2. Comparison of three models 26

4.3. Healthy-versus-diseased and species-level evaluation 28

4.4. Training curves 29

4.5. Class-level analysis 30

PHENIKAA UNIVERSITY

4.6. Ablation study 33

4.7. Computational efficiency 34

4.8. Calibration and selective prediction 36

4.9. Reference experiment with ImageNet pre-training 37

CHAPTER 5: ERROR ANALYSIS AND INTERPRETABILITY 39

5.1. Main Sources of Classification Errors 39

5.2. Suspect training labels 40

5.3. Background shortcut analysis 41

5.4. Summary of the Grad-CAM evidence 42

5.5. Example predictions 42

CHAPTER 6: CONCLUSION AND FUTURE WORK 44

7.1. Results 44

7.2. Evaluation 44

7.3. Discussion 44

7.4. Future work 45

REFERENCES 46

LIST OF FIGURES

Figure 1: The distribution of data classes in the training set. 13
Figure 2: Data sample. 15
Figure 3: The final Grad-CAM layer of ResNet18, DSCNet, and SE-ResNet18. 21
Figure 4: Comparison of the actual Grad-CAM with two controls (a Gaussian filter applied to the image and a Grad-CAM applied to a different image). 23
Figure 5: DSCNet's Grad-CAM in the final layer (5×5, center column) and the intermediate layer (10×10, right column). 24
Figure 6: Multi-seed comparison of ResNet18, SE-ResNet18 and DSCNet 27
Figure 7: Accuracy at three levels of granularity (27 classes, 13 species, healthy vs diseased) for the three models 29
Figure 8: Species-level and healthy/diseased confusion matrices of DSCNet (seed 42) 29
Figure 9: Training curves of DSCNet (seed 42) 30
Figure 10: Training curves of ResNet18 (seed 42) 30
Figure 11: Training curves of SE-ResNet18 (seed 42) 30
Figure 12: Normalized confusion matrix of DSCNet (seed 42, row-normalized recall). 31
Figure 13: Normalized confusion matrix of ResNet18 (seed 42) 31
Figure 14: Normalized confusion matrix of SE-ResNet18 (seed 42) 32
Figure 15: Per-class F1-score of DSCNet on the test set (seed 42). 33
Figure 16: Ablation of DSCNet: validation and test macro-F1 for each configuration. 34
Figure 17: GMACs, GPU latency versus batch size, and CPU latency (batch 1) of the three model 35
Figure 18: Reliability diagrams of the three models before and after temperature scaling 36
Figure 19: Trade-off between confidence threshold, coverage and accuracy of DSCNet (calibrated) 37
Figure 20: From-scratch models versus the ImageNet-pretrained ResNet18 reference 38
Figure 21: The 16 test images misclassified by DSCNet with the highest confidence (true label and predicted label with confidence) 39
Figure 22: Training images with the highest loss under DSCNet, with their given label and the model prediction 40

Figure 23: Accuracy under background blurring, background removal and leaf removal, compared with random-region controls (155 images) 42
Figure 24: Example test predictions of DSCNet with top-3 probabilities and the uncertainty flag (threshold 0.4) 43

PHENIKAA UNIVERSITY

LIST OF TABLES

Table 1: ResNet18 architecture with a 160×160×3 input. 17
Table 2: DSCNet architecture with 160×160×3 input. 20
Table 3: Multi-seed comparison of the three architectures (mean ± std over seeds 42, 1, 2). 26
Table 4: Detailed results of the seed-42 run with 95% bootstrap confidence intervals. 27
Table 5: Pairwise comparison on the test set (seed 42). 27
Table 6: Hierarchical metrics on the test set (mean ± std over three seeds). 28
Table 7: Ablation of DSCNet (mean ± std over three seeds). 34
Table 8: Computational cost and latency (Tesla T4 GPU and Colab CPU). 35
Table 9: Calibration of the three models on the test set (temperature fitted on the validation set). 37
Table 10: From-scratch models versus the pre-trained reference (mean ± std over three seeds). 38
Table 11: Background shortcut test on 155 test images (accuracy and accuracy drop). 41

CHAPTER 1: PROBLEM SITUATION AND PROBLEM DEFINITION

1.1. Problem situation

Agriculture is the main source of livelihood for a large share of the population in many developing countries. Plant diseases, however, threaten food quality and productivity: they are estimated to cost the global economy around US$220 billion annually, and in India alone about 35% of the annual crop yield is lost to plant diseases [1]. Early detection and accurate diagnosis of symptoms on plant leaves are therefore critical for timely intervention. Traditional diagnosis relies mainly on manual observation by agronomists, a process that is time-consuming, subjective and difficult to access for small-scale farmers in remote areas. Early detection and accurate diagnosis of disease symptoms on plant leaves are critical for timely intervention. However, traditional diagnostic methods rely primarily on manual observation by agronomists – a time-consuming process that is prone to subjectivity and difficult to access for the vast majority of small-scale farmers in remote areas.

The rapid advancement of Computer Vision combined with Deep Learning has opened up promising avenues for automating the process of crop disease diagnosis. Since the pioneering work by Mohanty et al. (2016) [2], which achieved over 99% accuracy on the PlantVillage dataset [3], research on leaf disease diagnosis has made significant strides. However, laboratory datasets such as PlantVillage are typically captured under ideal conditions with a monochromatic background and fixed lighting, leading to a severe drop in model performance when applied to real-world environments. This has driven a shift in research toward field datasets such as PlantDoc [1] – the primary focus of this study. In practice, applying deep learning models to the PlantDoc field data presents three major technical challenges:

- ***Natural environmental noise:*** Field images contain many complex sources of noise, such as the ground, the hand holding the leaf, shadows, surrounding tree branches, uneven lighting, and varied shooting angles.
- ***Class imbalance:*** In the original PlantDoc training split, class sizes range from 2 images ("Tomato two-spotted spider mites leaf") to 179 images ("Corn leaf blight"). After removing the two-image class (Section 2.3.1), the remaining 27 classes contain about 44 to 179 images (an imbalance ratio of about 4:1). This imbalance can still bias a CNN toward the majority classes and make it overlook minority classes..

- ***Computational resource constraints and real-time requirements:*** For the system to directly support farmers in the field, the diagnostic model cannot rely entirely on expensive GPU servers. Instead, the model must be lightweight enough to run directly on edge devices or handheld computers with limited computational power, delivering real-time inference speeds on a CPU (low latency) while maintaining reliable classification accuracy.

For these reasons, researching and designing a space-efficient CNN architecture – combined with standardized data processing workflows and robust data augmentation techniques – to enable real-time plant disease detection on the PlantDoc dataset is of the utmost urgency, both scientifically and in practical applications.

1.2. Problem definition

The problem in this study is formulated as a multi-class image classification problem. For each input leaf image with dimensions 160 × 160 × 3, the model must predict one of 27 corresponding plant or disease classes. The problem is designed not only to accurately identify the condition of the leaf but also to enable deployment in real-world conditions with low inference time and computational resource requirements.

- ***Input:*** A static leaf image of size H × W × 3, where the three channels correspond to the RGB color space. The image may be extracted from a camera feed or captured by a mobile device in natural environments, where complex backgrounds, uneven lighting, and other sources of noise may be present. During processing, the image is resized to 160 × 160 × 3 before being fed into the model.
- ***Output:*** The model returns a probability vector distributed across the 27 classes of the problem. Each element in the vector represents the model's confidence level for a corresponding class. The predicted label is selected as the class with the highest probability, thereby determining the healthy state or disease symptoms corresponding to the input image.
- ***Label Space:*** The set of labels Y consists of 27 distinct classes, including healthy leaf states and disease symptoms across 13 common crop species, namely: Apple, Bell Pepper, Blueberry, Cherry, Corn, Grape, Peach, Potato, Raspberry, Soybean, Squash, Strawberry, and Tomato. Mathematically, the system learns a mapping function f_θ, where θ is the set of learned weights of the neural network:

$$f_\theta : \mathbb{R}^{(H \times W \times 3)} \rightarrow [0, 1]^{27}$$

Specifically, given an input image x, the model generates a probability vector consisting of 27 elements. The predicted label is determined by:

$$\hat{y} = \arg\max f_\theta(x)$$

or equivalently, the label is selected as the class with the highest predicted probability among the 27 classes. Here, x is the input image and $\hat{y}$ is the model's predicted label. After preprocessing, the actual input size of the model is fixed at 160 × 160 × 3; therefore, the mapping function can be specifically expressed as:

$$f_\theta: \mathbb{R}^{(160 \times 160 \times 3)} \rightarrow [0, 1]^{27}$$

- ***Real-time Constraints:*** In addition to achieving good classification performance as measured by metrics such as accuracy and F1-score, the system is designed for real-time recognition applications. Therefore, the model must have low inference latency and appropriate hardware resource usage, particularly when deployed on CPUs or edge devices. This also serves as the basis for research that not only evaluates the classification capabilities of the models but also examines their number of parameters and computational efficiency.

PHENIKAA UNIVERSITY

CHAPTER 2: OBJECTIVES AND DATA PROCESSING

2.1. General objectives

This study aims to develop a deep learning system capable of automatically identifying and classifying the health status and diseases of plant leaves from images. The system was developed using the PlantDoc dataset, with the goal of classifying plant leaf images into 27 classes across various crop groups, including both healthy and diseased leaves. In addition to achieving accurate classification, the study aims to establish a data processing and model training workflow suitable for images collected under real-world conditions, where variations in lighting, shooting angles, and complex backgrounds may occur.

The study also focuses on developing and comparing various deep learning architectures – including ResNet18, SE-ResNet18, and DSCNet – to evaluate the balance between classification performance and computational cost. In particular, DSCNet is designed to reduce the number of parameters and computational cost through Depthwise Separable Convolution, thereby laying the groundwork for deploying the model on devices with limited resources. The effectiveness of the models is evaluated using various metrics, such as Accuracy, Macro-F1, Top-3 Accuracy, and stability across multiple experiments. Through this, the study aims to select an appropriate model and assess the system's applicability to the problem of plant leaf disease detection under real-world conditions.

2.2. Research objectives

This study aims to develop and evaluate a system for identifying and classifying plant leaf diseases from field images using deep learning, while also examining its feasibility for deployment under conditions requiring low inference time. Specifically, the study aims to achieve the following objectives:

- Develop a data processing pipeline for the PlantDoc dataset, including data cleaning, removal of underrepresented classes, splitting the training and validation sets using a stratified split method, image normalization, and the application of data augmentation techniques to improve the model's generalization ability.
- Building and training from scratch three CNN architectures – ResNet18, SE-ResNet18, and DSCNet – under the same experimental setup to enable a fair comparison of classification performance and computational cost.

- Design DSCNet using Depthwise Separable Convolution combined with residual connections to reduce the number of parameters and computational cost, aiming for fast inference on CPUs or edge devices.
- Evaluate the models using multiple metrics, including Validation Macro-F1, Test Accuracy, Test Top-3 Accuracy, Test Macro-F1, and the number of parameters.
- We conducted multi-seed evaluations and ablation studies to assess the stability of the results and determine the impact of data augmentation, class-weighted loss, and batch size on DSCNet's performance.
- We analyzed the model's decision interpretability using Grad-CAM and occlusion sensitivity analysis to verify whether the model focuses on regions containing disease symptoms rather than relying too heavily on the background.
- Conduct a ResNet18 experiment using pretrained weights from ImageNet as a reference experiment to evaluate the potential for performance improvement when applying transfer learning.

Select DSCNet as the most suitable model among the three architectures trained from scratch based on validation performance, test performance, and computational cost.

2.3. Data processing

2.3.1. Data cleaning

The original PlantDoc dataset exhibits a long-tailed distribution, which is very characteristic of data collected in the wild. While the majority class, "Corn leaf blight," contains 179 images, there are extremely rare minority classes such as "Tomato two-spotted spider mites," which has only 2 images in the training set and 0 images in the test set. This disparity results in a massive imbalance ratio of up to 89.5 times.

- ***Decision to remove the noise class:*** According to the learning principles of Convolutional Neural Networks (CNNs), updating weights via the backpropagation algorithm requires a minimum number of samples for the gradient to converge. With only two images, the model cannot generalize any morphological features of the red spider on the tomato leaf. Attempting to retain this class not only causes the algorithm to fail in classifying them but also acts as noise that distorts the loss landscape of the remaining 26 classes.

- ***Ensuring statistical reliability:*** Since this class has no images in the test set, retaining it also skews the macro evaluation metrics (Macro F1-Score, Macro Precision). Therefore, the "Tomato two-spotted spider mites" class was completely removed from the pipeline to ensure the rigor of the scientific experiment.

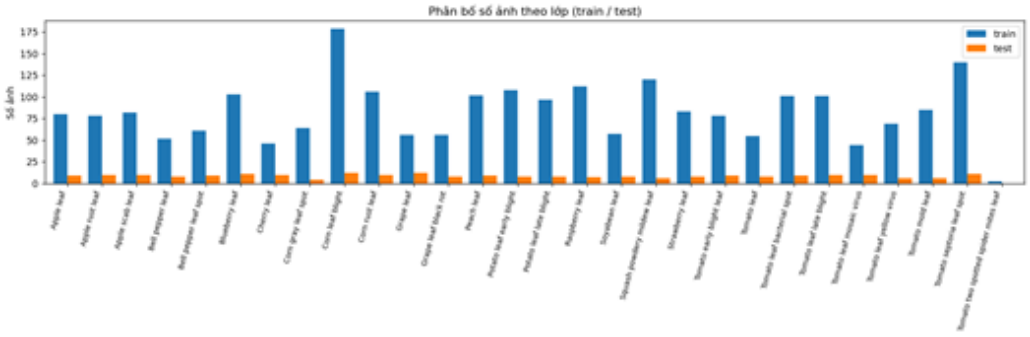


Figure 1: The distribution of data classes in the training set.

2.3.2. Data split

The data splitting strategy plays a decisive role in the model's generalization ability, especially on a small, high-noise dataset like PlantDoc.

- ***The 85/15 ratio was chosen:*** Instead of the traditional 70/30 or 80/20 ratios, to keep as many images as possible for training, the system uses an 85% Training set and a 15% Validation set. Due to the limited total number of images, prioritizing as much data as possible for the training set is crucial for the CNN to form sufficient feature maps. The 15% threshold is calculated as the minimum sufficient limit to generate a stable validation curve, supporting the Early Stopping technique to halt training as soon as overfitting begins to occur.
- ***Stratified Split:*** Faced with an class sizes that differ by about 4:1 (some classes have fewer than 50 training images), if a pure random split (Random Split) is applied, the minority classes risk being entirely assigned to the validation set and disappearing from the training set. Stratified Split thoroughly resolves this issue by forcing the algorithm to preserve the exact probability distribution P(Y) of each class in both the training and validation sets. This ensures the model is trained on all disease types and that the validation loss accurately reflects the model's ability to converge uniformly across the entire dataset.

- ***Absolute independence of the Test set:*** To ensure objectivity in scientific evaluation, the Test set (236 images) was kept intact from the original split by the PlantDoc authors. This dataset was held out throughout the weight update (backward pass) and hyperparameter tuning processes. This isolation completely eliminates the risk of data leakage, ensuring that Test Accuracy accurately reflects the model's performance when deployed in real-world settings.

2.3.3. Image preprocessing

Image preprocessing is a mandatory step applied uniformly to all three datasets: Train, Validation, and Test. This step aims to transform diverse images into a standardized vector space for input into the CNN. Specifically, it involves two basic operations:

- ***Unifying the image space (Resize to 160x160):*** Since the images in PlantDoc were collected from various devices (smartphones, digital cameras), they have vastly different resolutions and aspect ratios. Meanwhile, a CNN with a Fully Connected layer (or Global Average Pooling at the end) requires the input tensor size to be fixed. Resizing all images to a resolution of 160x160 pixels represents a delicate trade-off: It is large enough to preserve the spatial texture of necrotic lesions or microscopic deposits, while significantly reducing the number of floating-point operations (FLOPs). As a result, the new model is capable of reducing latency to below 33 ms, meeting the requirements for real-time inference on edge devices with limited computational resources.
- ***Z-score Normalization:*** Original RGB images have pixel values in the range [0, 255]. If these large values are fed directly into the network, the signal amplitude differences will cause the loss landscape to become extremely rugged and steep. In this case, the Gradient Descent algorithm will continuously oscillate and find it very difficult to converge to the global minimum. Using Z-score normalization based on the mean values (Mean=[0.485, 0.456, 0.406]) and standard deviations (Std=[0.229, 0.224, 0.225]) of the ImageNet dataset helps center the distribution of pixel values around the 0 axis with a variance of approximately 1. This is perfectly compatible with weight initialization algorithms such as Kaiming/He Initialization, helping to accelerate training and stabilize the gradient flow within the network.



Figure 2: Data sample.

2.3.4. Strong augmentation

This step applies only to the training set and is performed on the GPU, per image, on tensors scaled to [0, 1] before normalization. Because the training set contains only 1,944 images, an unregularized network memorizes it quickly. Each training image is transformed as follows:

- ***Random affine transform:*** Rotation, zoom and translation are applied in a single resampling step: the rotation angle is drawn uniformly from ±45°; the zoom is drawn so that the visible window covers 60–100% of the image side (36–100% of the area) and is resampled to 160×160; a random translation keeps the window inside the image; reflection padding fills the corners exposed by the rotation. This simulates different camera distances, angles and framing of the leaf.
- ***Random horizontal flip:*** with probability 0.5 (no vertical flip).
- **Photometric jitter*:*** Brightness, contrast and saturation are each multiplied by a random factor drawn uniformly from [0.7, 1.3], simulating strong sunlight, overcast sky and shadows. Hue is left unchanged.
- ***Random Erasing (p=0.10, scale=0.02~0.10):*** vertical flipping and grayscale conversion are not used. A weaker variant (rotation ±15°, zoom 85–100%, colour factors ±15%) was used only in the ablation study.

CHAPTER 3: ARCHITECTURAL MODELS AND METHODS

This study proposes and compares three convolutional network architectures: ResNet18 as the baseline model, SE-ResNet18, which incorporates a channel-wise attention mechanism, and DSCNet, a lightweight architecture designed specifically for this task. All three networks were trained from scratch with randomly initialized weights; no pre-trained weights were used. They all take 160×160 RGB images as input and output probability vectors across 27 layers. As a result, any performance differences between the models can be attributed to the architectures themselves, rather than differences in data or training procedures. This chapter presents each architecture in turn, followed by an introduction to the Grad-CAM technique, which is used to interpret the image regions that the models rely on when making predictions.

3.1. Model ResNet18

ResNet18 [4] was chosen as the baseline model because its structure has been extensively validated, is easy to reproduce, and facilitates comparison. The main challenge with deep networks is that gradients diminish as they propagate backward through multiple nonlinear layers, causing the early layers to learn very slowly. ResNet addresses this issue using skip connections. Instead of having a group of layers directly learn the mapping H(x), the network only needs to learn the residual F(x) = H(x) − x, and then add this residual to the input itself:

$$y = ReLU(F(x, \{W_i\}) + x)$$

Because the input x is added directly to the output, the gradient follows a path that does not pass through any nonlinear transformations. This helps make the network more stable during training, especially under from-scratch training conditions as in this study.

The basic unit of the network is the BasicBlock, consisting of two 3×3 convolutions, each accompanied by batch normalization (BN) [5], with a ReLU activation function placed between the two convolutions. For a single convolution, the value at position (i, j) of the output feature map is calculated as follows:

$$Y(i,j) = \sum_m \sum_n X(i, +m\, j + n) \cdot K(m,n) + b$$

Batch normalization ensures that the output of each convolution layer has a mean of 0 and a variance of 1 within each batch, and then allows the network to relearn the elasticity and shift:

$$\hat{x} = \frac{(x - \mu_B)}{\sqrt{\sigma_B^2 + \varepsilon}}$$

$$y = \gamma \cdot \hat{x} + \beta$$

Here, μ_B and σ_B^2 are the mean and variance of the batch, ε is a small constant to avoid division by zero, and γ and β are two learned parameters. When the number of channels or the spatial dimension of the output differs from that of the input, the dropout branch uses an additional 1×1 convolution followed by BatchNorm so that both branches have the same size before addition.

In terms of overall structure, the network begins with a stem block consisting of a 7×7 stride-2 convolution with 64 channels, followed by BatchNorm, ReLU, and a 3×3 stride-2 MaxPooling, aimed at rapidly reducing the resolution at the beginning of the network. The main body consists of four stages, each containing two BasicBlocks. The number of channels increases exponentially from 64 to 128, 256, and 512. The last three stages all have a stride of 2 in the first block to halve the spatial size. At the end of the network, the Global Average Pooling layer aggregates each feature map into a single value; the result passes through Dropout with a probability of 0.3, then into a linear layer (512 → 27) and the Softmax function. The convolution weights are initialized using the Kaiming method. The γ coefficient of the final BatchNorm layer in each block is set to 0, so in the early stages, each block acts almost as an identity transformation. This initialization method helps make the training process more stable from the start on small datasets.

Table 1: ResNet18 architecture with a 160×160×3 input.

Floor	***Output Size***	***Details***
Entrance	160×160×3	Normalized RGB image
Stem	40×40×64	7×7/2 Conv → BN → ReLU → 3×3/2 MaxPool
Stage 1	40×40×64	2 BasicBlocks, 64 channels
Stage 2	20×20×128	2 BasicBlocks, 128 channels, stride 2
Stage 3	10×10×256	2 BasicBlocks, 256 channels, stride 2
Stage 4	5×5×512	2 BasicBlocks, 512 channels, stride 2
Classification Head	1×27	GAP → Dropout (0,3) → Linear (512, 27) → Softmax

The network has 11.19 million parameters and requires 0.93 GMACs per image

3.2. Model SE-ResNet18

SE-ResNet18 retains the entire architecture of ResNet18 and only adds a Squeeze-and-Excitation block (Hu et al., 2018) [6] inside each BasicBlock. The rationale behind this design stems from the fact that feature channels do not play equal roles. In the case of field images of leaves, some channels reflect the texture of disease spots or changes in leaf tissue color, while

many others primarily encode the ground, hands, or shadows. The SE block allows the network to assign a weight to each channel based on global information from the entire image, thereby amplifying useful channels and attenuating less relevant ones.

The SE block operates in three steps. In the squeeze step, each feature map u_c of size H×W is summarized into a single value via global averaging:

$$z_c = \frac{1}{H \times W} \sum(h = 1)^H \sum(w = 1)^W u_{c(h,w)}$$

In the excitation step, the vector z passes through two fully connected layers with a bottleneck structure. The first layer reduces the number of dimensions from C to C/r, the second layer restores the dimensions to C, and the sigmoid function maps each weight to the range (0, 1):

$$s = \sigma\big(W_2 \cdot \delta(W_1 \cdot z)\big)$$

Here, δ is the ReLU function, σ is the sigmoid function, $W_1 \in \mathbb{R}^{\frac{C}{r} \times C}$, $W_2 \in \mathbb{R}^{C \times \frac{C}{r}}$, and the compression ratio r = 16. Finally, in the scaling step, each feature map is multiplied by the corresponding weight:

$$\tilde{x}_c = s_c \cdot u_c$$

In this network, the SE block is placed immediately after the second convolution of BasicBlock and before the addition with the skip branch. The formula for a block becomes:

$$y = ReLU\big(SE\big(F(x, \{W_i\})\big) + x\big)$$

PHENIKAA UNIVERSITY

This position keeps the shortcut branches unchanged, so ResNet's gradient propagation properties are preserved. The rest of the network – including the stem, the number of stages, the number of channels, the initialization method, and the classifier – is identical to ResNet18, so the two networks differ only in their channel-wise attention mechanism.

3.3. Model DSCNet

DSCNet is a custom architecture inspired by lightweight networks such as MobileNet [7], built entirely from depthwise separable convolutions (DSC) combined with skip connections. The design objectives are twofold. The first is to significantly reduce the number of parameters and computational cost so that the model can run on CPUs and edge devices. The second is to mitigate overfitting, as the training dataset consists of only 1,944 images – a very small number relative to the size of a network with over 11 million parameters.

The idea behind DSC is to break down a standard convolution into two simpler steps. For a standard convolution with kernel $D_K \times D_K$, input channel C_{in}, and output channel C_{out}, applied to a feature map of size $D_F \times D_F$, the computational cost is:

$$Standard_{Cost} = D_K^2 \cdot C_{in} \cdot C_{out} \cdot D_F^2$$

In DSC, the first step is depthwise convolution, in which each channel is convolved with a separate 3×3 filter and the channels do not mix with one another:

$$\hat{G}(m,n,l) = \sum_h \sum_w X(m,+,h\,n + w\,l) \cdot K_{dw(h,w,l)}$$

This step processes spatial information – that is, it identifies local shapes and structures such as disease spots or leaf veins – at a cost of $D_K^2 \cdot C_{in} \cdot D_F^2$. The second step is a 1×1 pointwise convolution, which blends information across channels:

$$Y(m,n,k) = \sum_l \hat{G}(m,n,l) \cdot K_{pw(l,k)}$$

This step processes channel information – such as combining color with texture – at a cost $C_{in} \cdot C_{out} \cdot D_F^2$. The cost ratio between DSC and standard convolution is:

$$\frac{(D_K^2 \cdot C_{in} \cdot D_F^2 + C_{in} \cdot C_{out} \cdot D_F^2)}{(D_K^2 \cdot C_{in} \cdot C_{out} \cdot D_F^2)} = \frac{1}{C_{out}} + \frac{1}{D_K^2}$$

With a 3×3 kernel and a large number of output channels, this ratio is approximately 1/9. Thus, each DSC layer saves about 8 to 9 times the number of computations compared to a standard convolution of the same size.

The basic unit of the network is the DSBlock, which consists of a 3×3 depthwise convolution, BatchNorm, ReLU6, a 1×1 pointwise convolution, BatchNorm, and ReLU6 [8]. When the block retains the same stride and number of channels, the input is directly added to the output, as in ResNet; conversely, the block has no shortcut branches:

$$y = x + DSC(x)$$

when the step size is 1 and $C_{in} = C_{out}$; $y = DSC(x)$ in all other cases.

The entire network begins with a standard 3×3 convolution with a stride of 2 and 32 channels, followed by a sequence of 11 DSBlocks. The number of channels gradually increases from 64 to 1024, while the spatial resolution gradually decreases from 80×80 to 5×5 through five blocks with a stride of 2. Six blocks retain their original shape and therefore have shortcut branches. At the end of the network, the classification head consists of Global Average Pooling, Dropout with a probability of 0.3, and a linear layer mapping from 1024 to 27.

Table 2: DSCNet architecture with 160×160×3 input.

Stage	***Blocks (input channel → output channel, step size)***	***Output size***
Stem	Conv 3×3: 3 → 32, step 2	80×80×32
Group 1	DSBlock 32 → 64 (1); DSBlock 64 → 128 (2)	40×40×128
Group 2	DSBlock 128 → 128 (1); DSBlock 128 → 256 (2)	20×20×256
Group 3	DSBlock 256 → 256 (1); DSBlock 256 → 512 (2)	10×10×512
Group 4	DSBlock 512 → 512 (1) × 3; DSBlock 512 → 1024 (2)	5×5×1024
Group 5	DSBlock 1024 → 1024 (1)	5×5×1024
Classification Header	GAP → Dropout (0,3) → Linear (1024, 27) → Softmax	1×27

The network has only 2.70 million parameters – about 4.1 times fewer than ResNet18 – and requires 0.24 GMACs per image, which is about 3.9 times lower. This efficiency is clearly evident in real-world performance. CPU latency is only 11.17 ms per image, which is about 2.6 times faster than ResNet18. On the GPU, latency is 2.76 ms for a batch size of 1 and 0.18 ms per image for a batch size of 32.

In terms of accuracy, DSCNet achieves an average Test Accuracy of 37.3% ± 2.1% across three training runs with different seeds, which is significantly higher than that of ResNet18 (25.8% ± 1.5%) and SE-ResNet18 (26.6% ± 1.7%). In pairwise comparisons on the same test set, the McNemar test yielded p = 0.0005 between DSCNet and each of the other networks, while the two ResNet networks were not statistically significantly different (p = 1.0). A plausible explanation is that smaller networks tend to retain less training data when the sample size is limited. Additionally, DSC's separation of spatial processing from channel processing aligns with the characteristics of leaf disease images, where both the shape of the lesions and changes in color must be recognized.

3.4. Explanatory analysis using Grad-CAM

Overall accuracy alone does not reveal which features the model relies on to make decisions. With field data, a network may achieve decent results by using "shortcuts" [9] such as background color or image style rather than examining disease lesions. Therefore, this study uses Grad-CAM [10] to visualize the image regions that most influence each model's predictions.

Given an input image and a prediction class c, Grad-CAM first calculates the importance weights of each channel k in the feature map A^k of the target convolution layer by averaging the gradients of the score y^c across that map:

$$\alpha_k^c = \frac{1}{Z}\sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}$$

A heatmap is a linear combination of feature maps weighted by the above weights, retaining only the positive components:

$$L^c = ReLU\left(\sum_k \alpha_k^c \cdot A^k\right)$$

Here, Z is the number of spatial positions in the feature map, and the ReLU activation function removes regions that negatively affect class c. The map is bilinearly interpolated to 160×160, normalized to the range [0, 1] for each image, and then overlaid onto the original image.

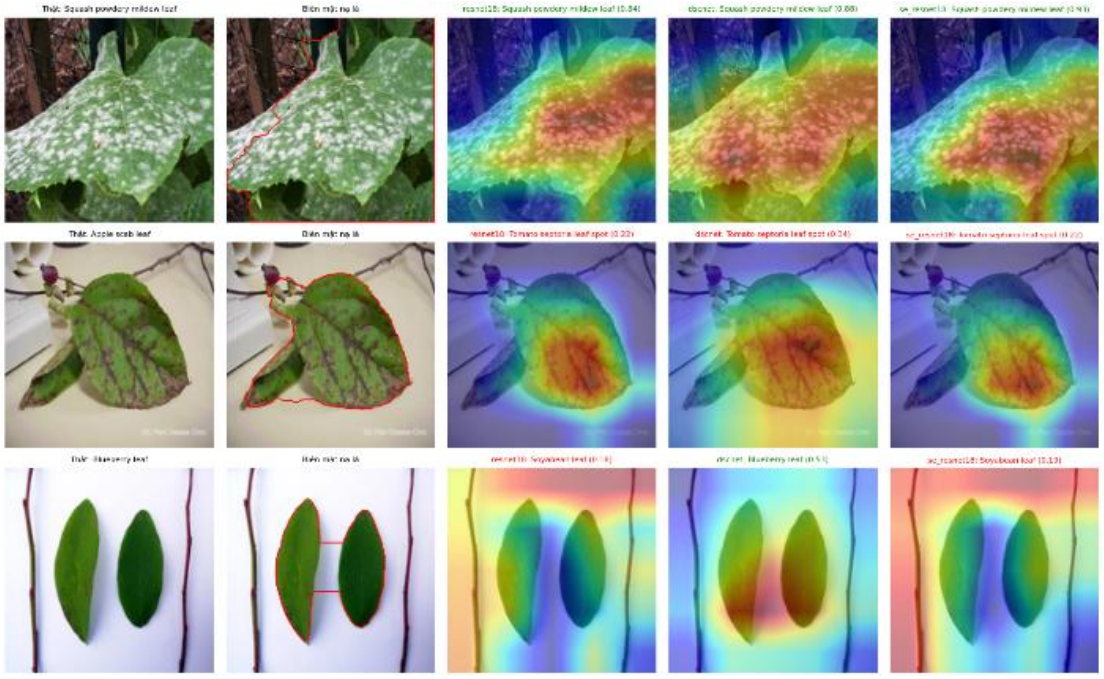


Figure 3: The final Grad-CAM layer of ResNet18, DSCNet, and SE-ResNet18.

Grad-CAM is computed at two locations within the network. The first location is the final convolutional layer – that is, the final residual block of Stage 4 for ResNet18 and SE-ResNet18, and the 11th DSBlock for DSCNet. The feature map here is 5×5 in size. The second position is an intermediate layer with a 10×10 map, specifically the final residual block of Stage 3 for the two ResNet networks, and the 9th DSBlock for DSCNet. The 10×10 map has twice the spatial resolution, allowing the highlighted regions to better capture smaller lesions. The 5×5 map from the final layer only shows a few large hotspots, which are rather coarse.

Qualitatively, Figure 3 compares the final-layer Grad-CAMs of the three models on the same test images, including both correct and incorrect predictions.Observing the image above reveals

several trends. In the image showing damage that appears as white powder on a squash leaf, the hotspots of all three models are located on the damaged portion of the leaf, and all three models made correct predictions. In the image of an apple leaf with scab, the hotspots are also located on the portion of the leaf with brown spots, but all three models incorrectly classified it as Septoria leaf spot on tomatoes. Thus, in this case, the error stems from the fact that the two diseases have visually similar symptoms, not because the models focused on the background.

Conversely, in the image of blueberry leaves with a light background, the hotspots of ResNet18 and SE-ResNet18 are located in the background area above and between the two leaves, and these two models made incorrect predictions, while DSCNet focused on the leaf stem and made the correct prediction. Additionally, in images with complex backgrounds such as tree branches, soil, and overlapping leaves (e.g., images of tomato leaves affected by yellow leaf virus), the hotspots are scattered, and all three models make incorrect predictions. These examples are illustrative only, as they consist of just a few images; therefore, they should be cross-referenced with the quantitative results below.

To avoid drawing conclusions based solely on impressions from a few images, the study included a quantitative evaluation. Leaf region masks were automatically generated using color segmentation in HSV color space combined with GrabCut [11]. Mask quality was visually inspected on 40 random images, of which 34 (85%) met the requirements. After excluding images with incorrect masks, quantitative analysis was performed on 155 of the total 236 test images. This is a limitation of this experiment.

Three metrics were used: the Grad-CAM energy enrichment level in the leaf region, the enrichment level in the disease lesion region (estimated from the mask), and the proportion of images where the brightest spot is located within the leaf. An enrichment level greater than 1 indicates that the map focuses more on that region than on an even distribution. Two control experiments were used to rule out random effects. The first control was a Gaussian map centered on the image, simulating the photographer's tendency to place the leaf in the center of the frame. The second control was the Grad-CAM of a different image overlaid on the mask of the image under consideration. If the true Grad-CAM is content-dependent, it should outperform the second control.

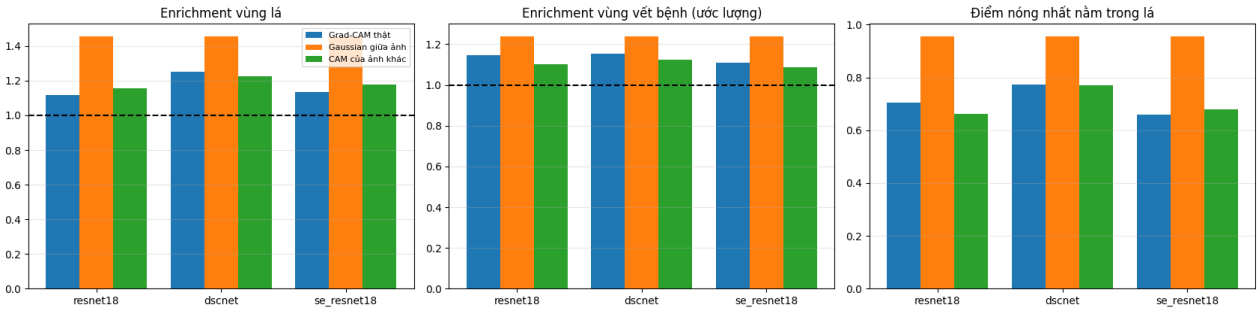


Figure 4: Comparison of the actual Grad-CAM with two controls (a Gaussian filter applied to the image and a Grad-CAM applied to a different image).

This figure shows that in the final layer, all three models have enrichment levels in the leaf region greater than 1 (ranging from approximately 1.12 to 1.25), but the Gaussian map of the center of the image is even higher (1.455) across all three metrics. This means that part of the observed enrichment may simply reflect the fact that leaves are often located in the center of the image. More importantly, the Grad-CAM results are virtually indistinguishable from those of other images. The difference between the two types in the leaf region enrichment metric was −0.040 for ResNet18, +0.025 for DSCNet, and −0.043 for SE-ResNet18, and all 95% confidence intervals contained the value 0. Therefore, the final-layer maps do not provide sufficient evidence to confirm that the model is focusing on disease spots.

At the intermediate layer, the results differ. Figure 5 directly compares the 5×5 and 10×10 maps of DSCNet on the same set of images. In images showing clear damage, such as scorched corn leaves, the 10×10 map is more precisely confined to the lesions, while the 5×5 map is more widely distributed. However, this map is not always consistent. In the image of a tomato leaf with a handprint, the hot spot falls in the corner of the image. Quantitatively, DSCNet at the intermediate layer achieves a higher leaf region enrichment level than Grad-CAM by +0.127 (95% confidence interval: +0.014 to +0.241), and a higher proportion of the hottest spots located within the leaf by +0.160 (+0.087 to +0.228). The two ResNet models showed virtually no difference from the baseline in the leaf region enrichment index (−0.009 and −0.004, with both confidence intervals containing 0). Regarding the disease spot region index, no model showed a statistically significant difference.

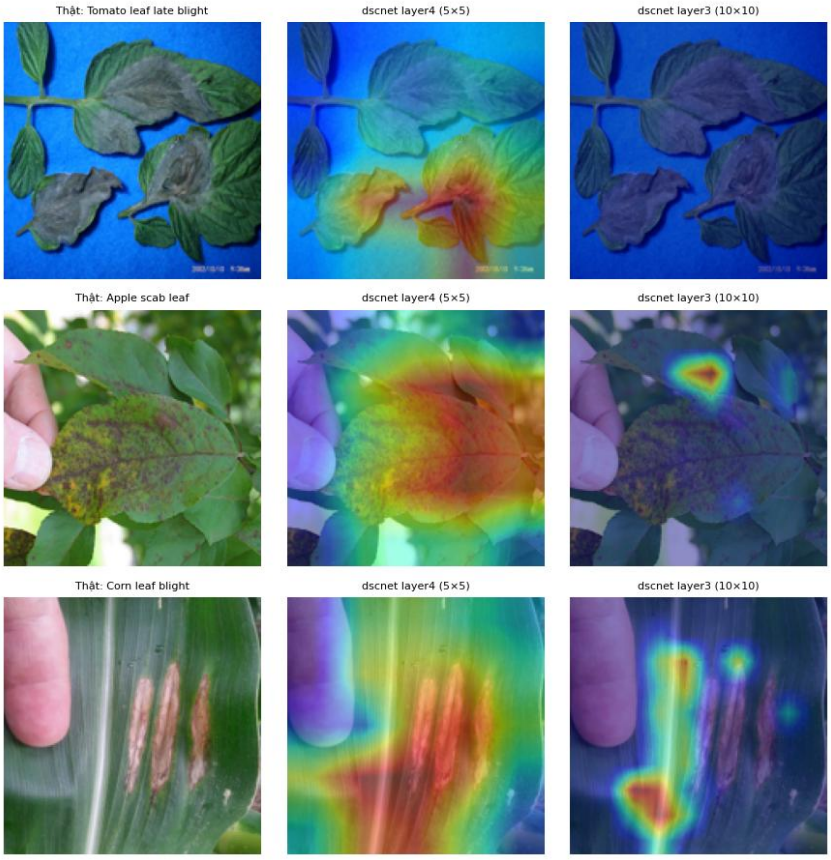


Figure 5: DSCNet's Grad-CAM in the final layer (5×5, center column) and the intermediate layer (10×10, right column).

The results of the region-masking experiment support this finding. We masked the 20% area with the highest Grad-CAM scores, then remeasured accuracy and compared it to masking a region of the same shape but placed at a random location. If the hotspots were truly important, masking the correct region should have caused a greater drop in accuracy. For DSCNet, accuracy after masking the hotspot was lower than with random masking by +0.129 (+0.086 to +0.173) at the final layer and +0.153 (+0.103 to +0.205) at the intermediate layer. With ResNet18, this effect is virtually nonexistent at the final layer (+0.012; the confidence interval contains 0) and appears only at the intermediate layer (+0.080). Thus, the regions identified by Grad-CAM truly influence DSCNet's decisions, more so than in ResNet networks.

Overall, DSCNet tends to extract features from leaf regions better than ResNet18 and SE-ResNet18, particularly in the intermediate layers. However, the results are only moderate for three reasons. First, the leaf mask coverage was only 85% on the test set. Second, the analysis dataset consisted of only 155 images. Third, at the final layer, none of the three models could be

distinguished from the control group. Therefore, this report does not conclude that the model "accurately identifies disease spots," but only concludes that DSCNet captures image content better than the other models under the current experimental conditions.

EEE
PHENIKAA UNIVERSITY

CHAPTER 4: RESULTS AND EVALUATION

4.1. Evaluation protocol

All three architectures were trained from scratch under an identical protocol: AdamW [12] (peak learning rate 2×10^{-3}, weight decay 5×10^{-2}), cosine learning-rate decay with warm-up, batch size 64, label smoothing 0.1 [13], class-weighted cross-entropy, at most 40 epochs, and early stopping on validation macro-F1 (patience 15). Every configuration was repeated with three random seeds (42, 1, 2). The seed changes weight initialization and the order and augmentation of mini-batches, while the train/validation/test split is fixed. The main metrics are validation macro-F1, test accuracy, test Top-3 accuracy and test macro-F1 on the 236 test images. Results are reported as mean ± standard deviation over seeds. For seed 42, 95% confidence intervals were obtained by bootstrap resampling of the test set (1,000 resamples) [14], and pairwise model differences were tested with the exact McNemar test [15].

4.2. Comparison of three models

Table 3 summarizes the multi-seed results and Figure 6 visualizes them. DSCNet achieved the best result on every metric: a test accuracy of 37.3% ± 2.1%, a Top-3 accuracy of 62.1% ± 1.1% and a test macro-F1 of 0.354 ± 0.016, compared with 25.8% ± 1.5% / 0.249 ± 0.019 for ResNet18 and 26.6% ± 1.7% / 0.252 ± 0.013 for SE-ResNet18. The ranking is stable: DSCNet is the best model in all three seeds (test accuracy 0.373, 0.352 and 0.394 versus at most 0.284 for the ResNet-based models). DSCNet also has the highest mean validation macro-F1 (0.372), so it was selected as the best model using validation data only, not the test set:

Table 3: Multi-seed comparison of the three architectures (mean ± std over seeds 42, 1, 2).

Model	***Val macro-F1***	***Test Acc (%)***	***Test Top-3 (%)***	***Test macro-F1***	***Params (M)***	***GMACs***
ResNet18	0.288 ± 0.011	25.8 ± 1.5	52.5 ± 2.6	0.249 ± 0.019	11.19	0.93
SE-ResNet18	0.285 ± 0.014	26.6 ± 1.7	53.8 ± 2.6	0.252 ± 0.013	11.28	0.93
DSCNet	0.372 ± 0.011	37.3 ± 2.1	62.1 ± 1.1	0.354 ± 0.016	2.70	0.24

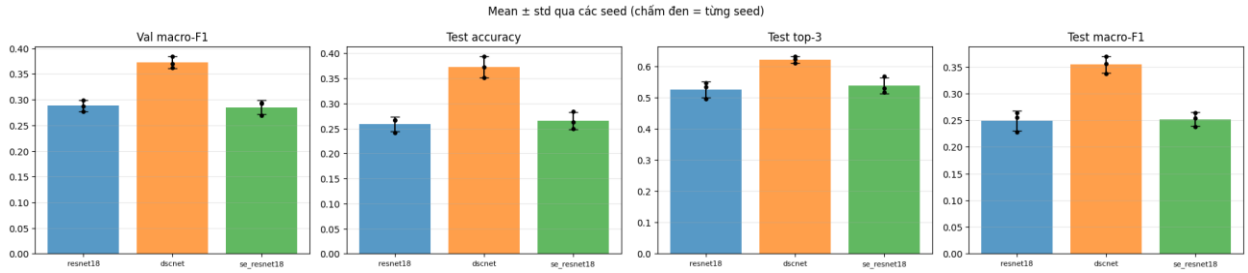


Figure 6: Multi-seed comparison of ResNet18, SE-ResNet18 and DSCNet

Table 4 gives the seed-42 run in detail, which is also the run used for the learning curves, confusion matrices and Grad-CAM analysis. The 95% bootstrap intervals of DSCNet and the two ResNet-based models overlap only partly, and the paired tests in Table 5 confirm the difference. DSCNet is better than ResNet18 by +10.6 points of accuracy (95% CI [5.1, 16.5]) and better than SE-ResNet18 by +11.0 points (CI [5.5, 17.0]); both McNemar tests give p = 0.0005, which remains significant after a Bonferroni correction for three comparisons (threshold 0.0167).

Table 4: Detailed results of the seed-42 run with 95% bootstrap confidence intervals.

Metric	***ResNet18***	***SE-ResNet18***	***DSCNet***
Val macro-F1	0.299	0.294	0.370
Test accuracy	0.267 [0.212, 0.322]	0.263 [0.208, 0.318]	0.373 [0.309, 0.436]
Test Top-3 accuracy	0.547	0.530	0.610
Test macro-F1	0.255 [0.195, 0.296]	0.254 [0.193, 0.295]	0.355 [0.288, 0.400]
Test weighted-F1	0.252	0.249	0.359
Best epoch	37	35	35
Training time (T4)	1.0 min	1.3 min	1.6 min

Table 5: Pairwise comparison on the test set (seed 42).

A − B	***ΔAcc [95% CI]***	***ΔF1 [95% CI]***	***A correct / B wrong***	***A wrong / B correct***	***McNemar p***
DSCNet − ResNet18	+0.106 [0.051, 0.165]	+0.098 [0.044, 0.154]	37	12	0.0005
DSCNet − SE-ResNet18	+0.110 [0.055, 0.170]	+0.099 [0.047, 0.152]	40	14	0.0005
ResNet18 − SE-ResNet18	+0.004 [−0.034, 0.042]	+0.000 [−0.035, 0.037]	11	10	1.000

In contrast, ResNet18 and SE-ResNet18 are statistically indistinguishable ($\Delta = +0.4$ points, $p = 1.0$): the channel-attention modules did not help in this small-data, from-scratch setting.

Top-3 accuracy is about 25 points above Top-1 accuracy for DSCNet (62.1% versus 37.3%). In many cases the correct class is therefore among the three most probable labels even when the top prediction is wrong. Macro-F1 is slightly lower than accuracy for all models, which reflects the weak performance on the rarest classes. Absolute accuracy remains low for all three models; Section 4.9 shows that this is mainly a data limitation rather than an architectural one.

4.3. Healthy-versus-diseased and species-level evaluation

Besides the 27-class accuracy, two coarser views of the same predictions were evaluated: (i) the plant species (13 classes) and (ii) a binary healthy/diseased decision (the test set contains 90 healthy and 146 diseased images, so always predicting "diseased" gives 61.9%).

Table 6: Hierarchical metrics on the test set (mean ± std over three seeds).

Model	***Exact acc.***	***Species acc.***	***Health acc.***	***Sensitivity***	***Specificity***	***Balanced acc.***	***Exact acc. given correct species***
ResNet18	0.258 ± 0.015	0.411 ± 0.033	0.866 ± 0.006	0.872 ± 0.022	0.856 ± 0.019	0.864 ± 0.002	0.630 ± 0.016
SE-ResNet18	0.266 ± 0.017	0.398 ± 0.017	0.869 ± 0.021	0.874 ± 0.034	0.859 ± 0.034	0.867 ± 0.020	0.666 ± 0.015
DSCNet	0.373 ± 0.021	0.531 ± 0.019	0.894 ± 0.017	0.893 ± 0.016	0.896 ± 0.026	0.894 ± 0.018	0.702 ± 0.015

Table 6 shows that all models detect the presence of disease much better than they identify the exact class. DSCNet reached 89.4% ± 1.7% healthy/diseased accuracy (sensitivity 0.893, specificity 0.896), versus 86.6% and 86.9% for ResNet18 and SE-ResNet18, and 28 points above the majority baseline. Species recognition is much weaker (53.1% for DSCNet), and when the species is predicted correctly the exact class is right in 70.2% of the cases. The value of 91.1% obtained by DSCNet for seed 42 is the best of the three seeds; the mean over seeds (89.4%) should

be used as the headline figure. These results indicate that the system is more reliable as a screening tool ("does this leaf need inspection?") than as an exact diagnostic tool.

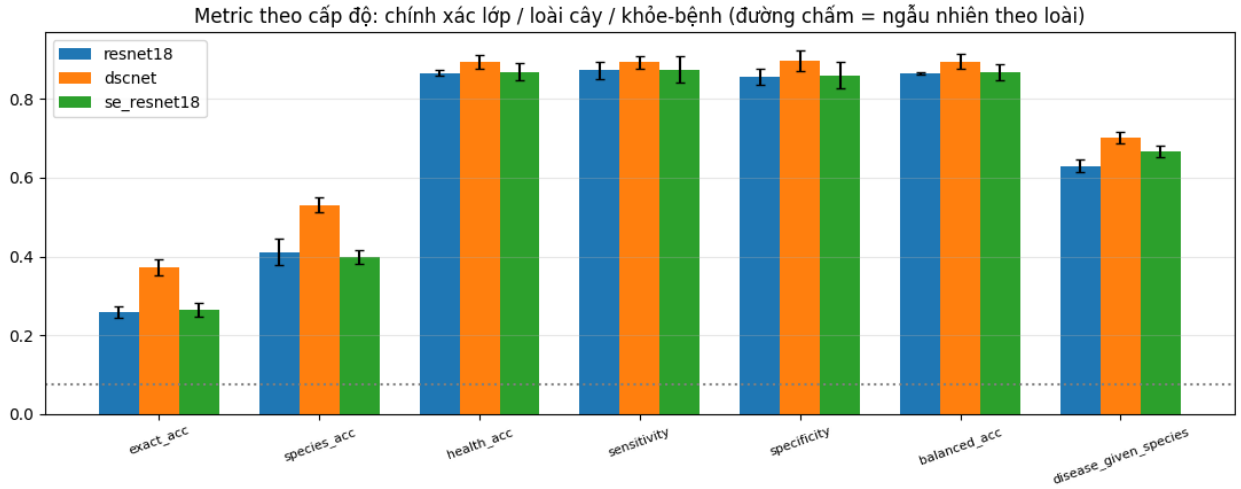


Figure 7: Accuracy at three levels of granularity (27 classes, 13 species, healthy vs diseased) for the three models

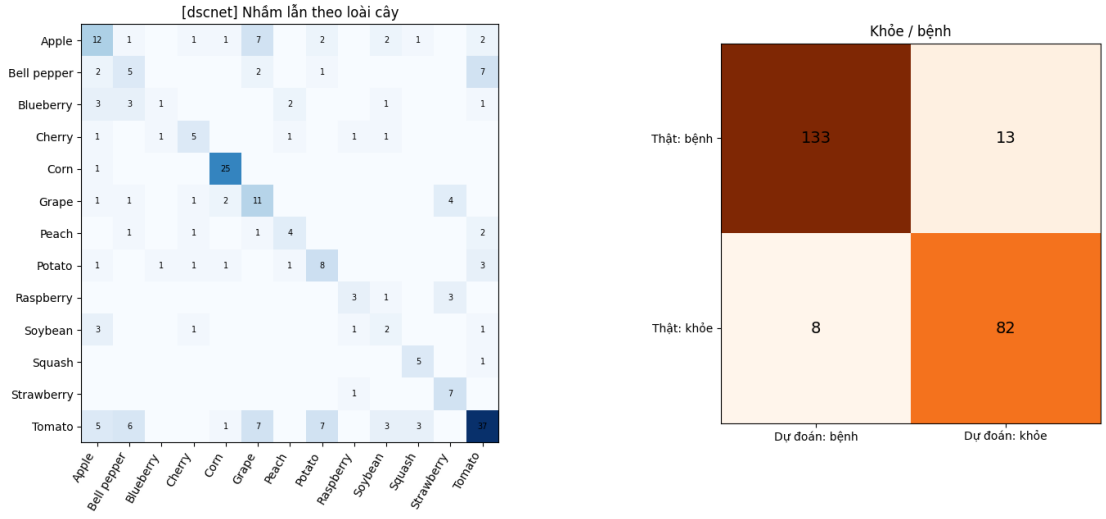


Figure 8: Species-level and healthy/diseased confusion matrices of DSCNet (seed 42)

4.4. Training curves

Figures 9 – 11 show the loss, accuracy, validation macro-F1 and learning-rate schedule of the seed-42 runs; the vertical line marks the best epoch. In all three runs the losses decrease steadily while the learning rate follows the cosine decay, and the best checkpoint is reached late in training (epoch 37 for ResNet18, 35 for DSCNet and 35 for SE-ResNet18), when the learning rate is already small. No run triggered early stopping before the 40-epoch limit, which suggests that

longer training could still bring small gains. The validation curves are noisy because the validation set contains only 344 images. All models show a gap between training and validation accuracy, as expected for a training set of 1,944 images; for DSCNet the mean gap at the best epoch is 0.171 ± 0.045 over three seeds (Section 4.6). DSCNet reaches clearly higher validation macro-F1 than the other two models throughout the second half of training.

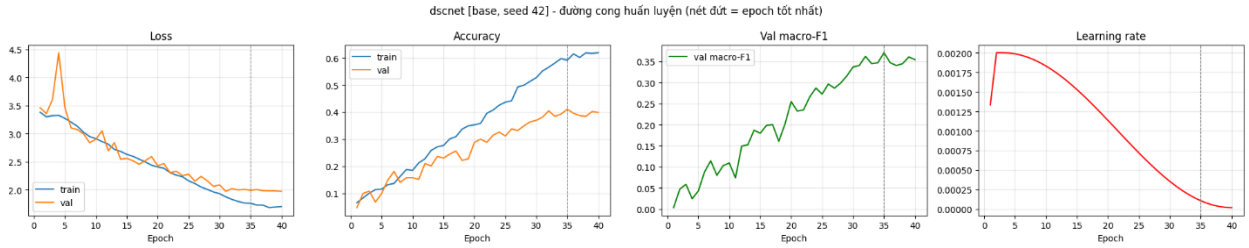


Figure 9: Training curves of DSCNet (seed 42)

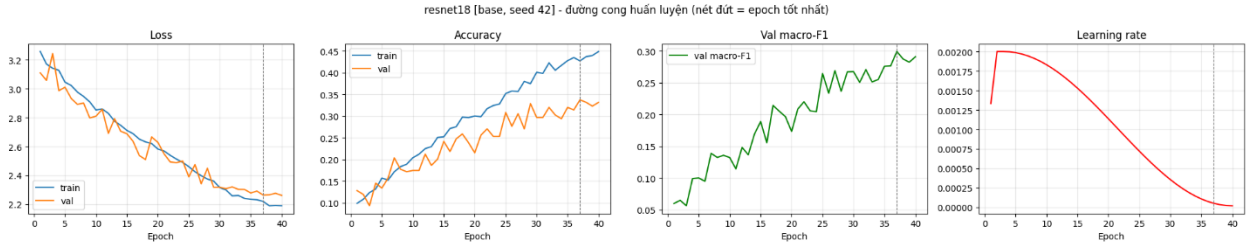


Figure 10: Training curves of ResNet18 (seed 42)

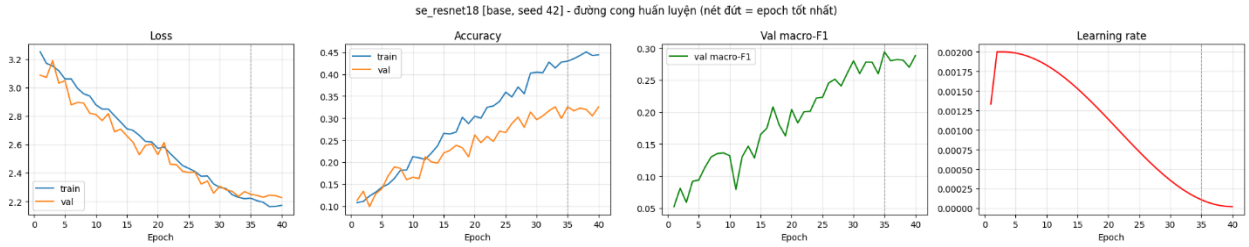


Figure 11: Training curves of SE-ResNet18 (seed 42)

4.5. Class-level analysis

Figures 12 – 14 show the normalized confusion matrices (27 classes) and Figure 15 the per-class F1 of DSCNet. DSCNet has a more pronounced diagonal than the two ResNet-based models, whose errors are spread over many classes. The best-recognized classes of DSCNet are Corn rust leaf (F1 0.762), Corn leaf blight (0.692), Squash powdery mildew leaf (0.667), Grape leaf (0.636) and Strawberry leaf (0.636), all classes with distinctive texture or colour. Two classes were never recognized by DSCNet (F1 = 0): Tomato leaf late blight and Tomato mold leaf.

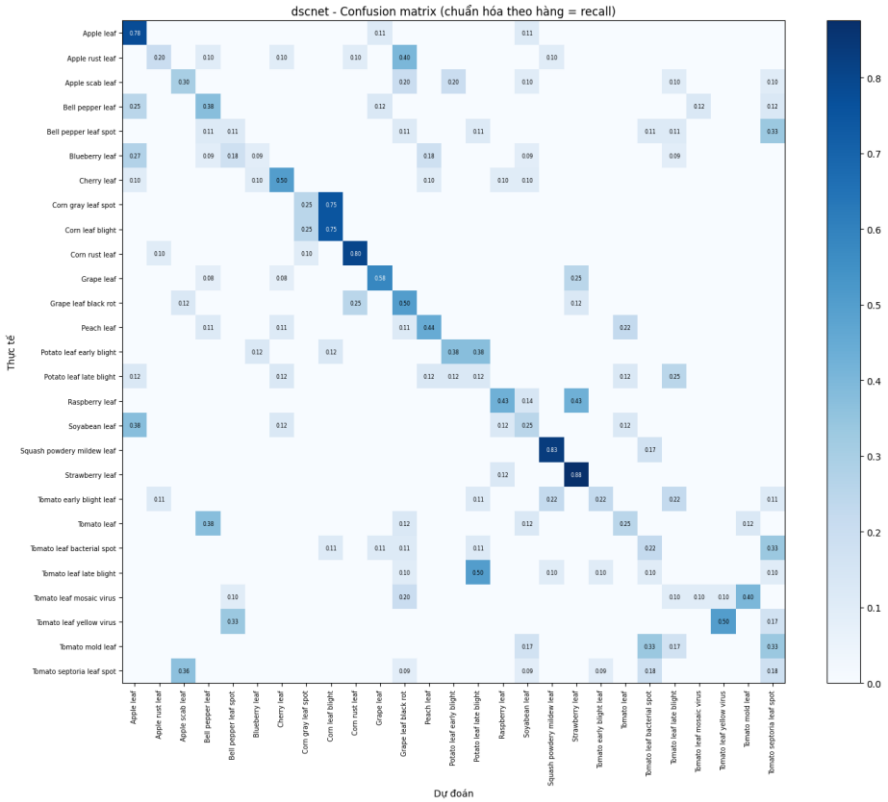


Figure 12: Normalized confusion matrix of DSCNet (seed 42, row-normalized recall).

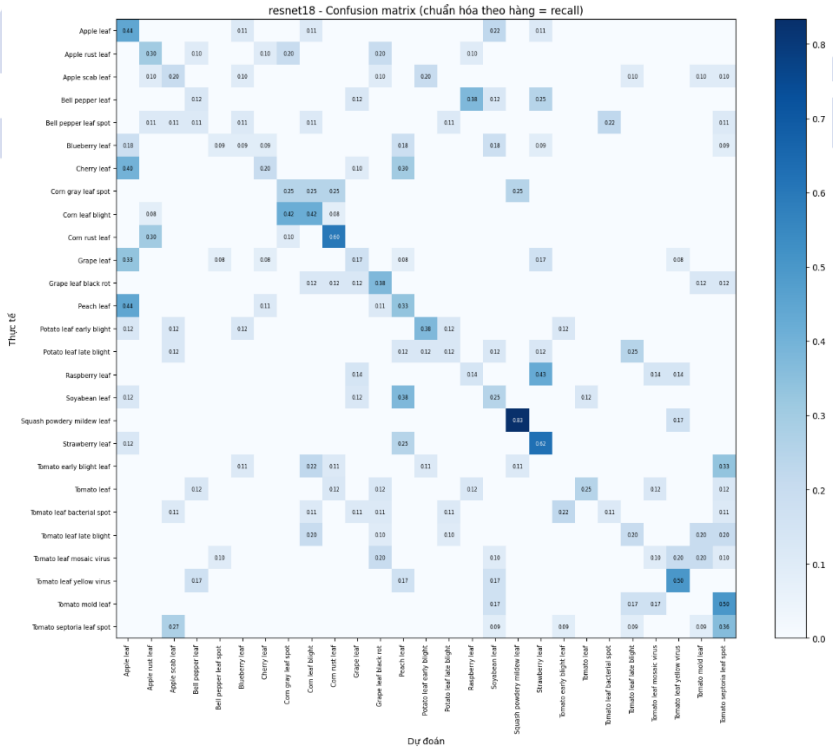


Figure 13: Normalized confusion matrix of ResNet18 (seed 42)

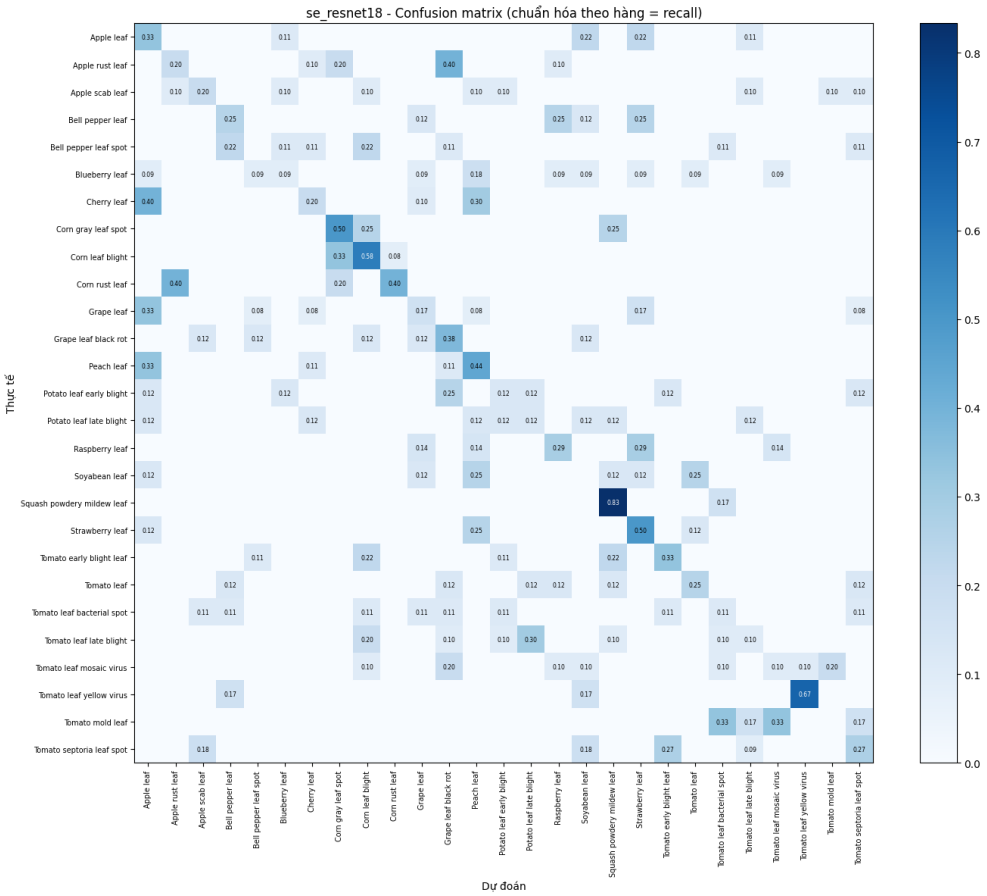


Figure 14: Normalized confusion matrix of SE-ResNet18 (seed 42)

The zero-F1 classes are Bell pepper leaf spot, Tomato early blight leaf and Tomato mold leaf for ResNet18, and Bell pepper leaf spot, Soyabean leaf and Tomato mold leaf for SE-ResNet18. Tomato mold leaf is not recognized by any model, and per-class values should be read with care since each class has only 4–12 test images.

The most frequent confusions (true → predicted, seed 42) are:

- DSCNet: Tomato late blight → Potato late blight (5 images); Apple rust → Grape black rot (4); Tomato mosaic virus → Tomato mold (4); Tomato septoria → Apple scab (4); Potato early blight → Potato late blight (3).
- ResNet18: Corn leaf blight → Corn gray leaf spot (5); Grape, Peach and Cherry leaf → Apple leaf (4 each).
- SE-ResNet18: Apple rust → Grape black rot (4); Cherry leaf → Apple leaf (4); Corn leaf blight → Corn gray leaf spot (4); Corn rust → Apple rust (4).

Two kinds of confusion therefore dominate: diseases with similar symptoms (blight on potato and tomato, corn leaf spots) and, for the ResNet-based models, healthy leaves of different species (Grape, Peach, Cherry → Apple).

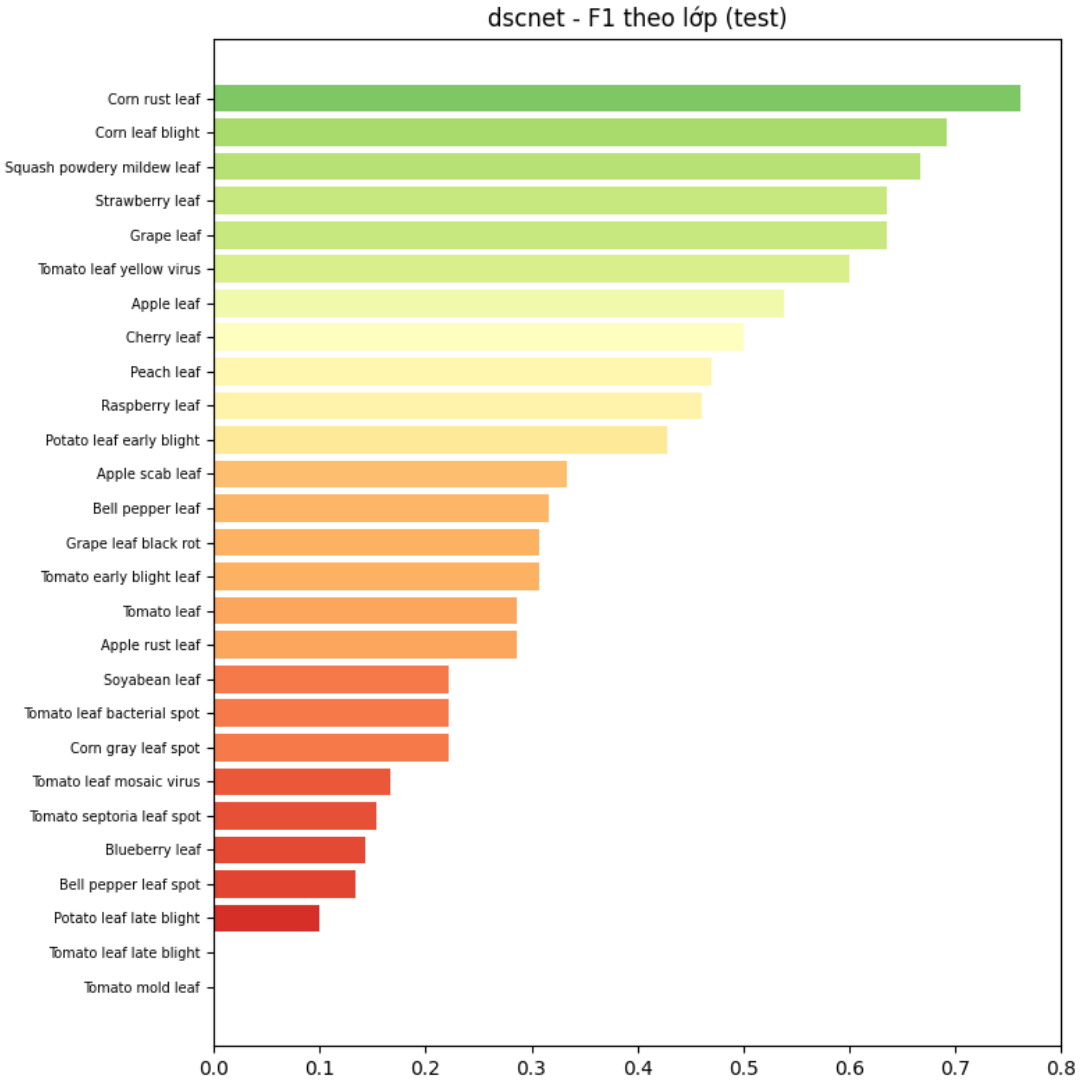


Figure 15: Per-class F1-score of DSCNet on the test set (seed 42).

4.6. Ablation study

The ablation study used DSCNet with the same three seeds (Table 7, Figure 16). Data augmentation is by far the most important component. Removing it reduces test macro-F1 from 0.354 to 0.144 and test accuracy from 37.3% to 16.4%, while the gap between training and validation accuracy rises from 0.171 to 0.792, that is, the network memorizes the training set. A weaker augmentation (smaller rotation, scale and colour range) is also worse: the test macro-F1

drops to 0.316 and the gap to 0.508, although the validation macro-F1 is identical (0.372), which shows that the small validation set cannot always resolve differences that the test set reveals.

The other changes did not bring a clear benefit. Removing the class weights gave 0.362 ± 0.015 test macro-F1 (+0.008), a difference smaller than the seed-to-seed deviation, so the weighted loss has no measurable effect here. A batch size of 32 gave 0.369 ± 0.005 (+0.015), and combining it with unweighted loss gave 0.370 ± 0.015 (+0.016, better than the baseline in all three seeds on the test set but in only two of three on validation). These gains are about one standard deviation of the baseline, so they are reported as a tendency and not adopted; the baseline configuration was kept for all other experiments.

Table 7: Ablation of DSCNet (mean ± std over three seeds).

Configuration	***Val macro-F1***	***Test Acc***	***Test macro-F1***	***Train–val gap***
base (strong aug., class weights, batch 64)	0.372 ± 0.011	0.373 ± 0.021	0.354 ± 0.016	0.171 ± 0.045
no augmentation	0.173 ± 0.014	0.164 ± 0.034	0.144 ± 0.028	0.792 ± 0.020
weak augmentation	0.372 ± 0.014	0.332 ± 0.016	0.316 ± 0.009	0.508 ± 0.019
no class weights	0.385 ± 0.005	0.388 ± 0.011	0.362 ± 0.015	0.200 ± 0.030
batch size 32	0.381 ± 0.019	0.381 ± 0.008	0.369 ± 0.005	0.154 ± 0.012
batch 32, no class weights	0.397 ± 0.038	0.393 ± 0.015	0.370 ± 0.015	0.155 ± 0.042

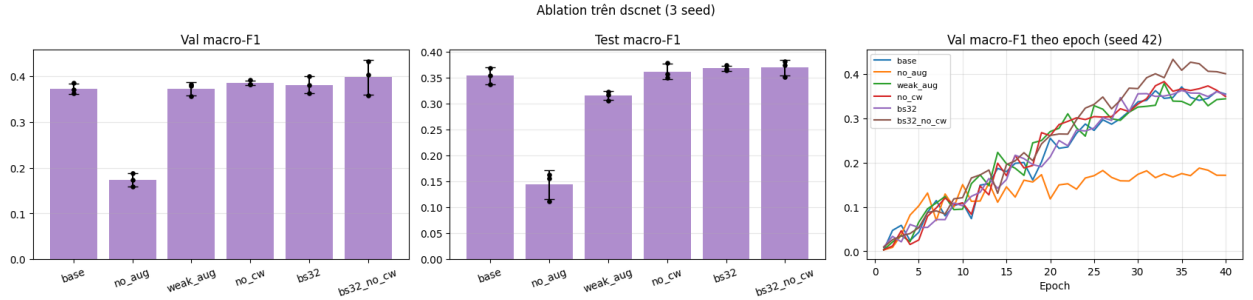


Figure 16: Ablation of DSCNet: validation and test macro-F1 for each configuration

4.7. Computational efficiency

Table 8 and Figure 17 compare the cost of the models. DSCNet has 4.1× fewer parameters (2.70M versus 11.19M) and 3.9× fewer multiply-accumulate operations (0.24 versus 0.93 GMACs) than ResNet18. This translates into a CPU latency of 11.2 ms per image at batch size 1, 2.6× faster than ResNet18 (29.3 ms) and SE-ResNet18 (29.7 ms). On the GPU the picture is

different: at batch size 1, DSCNet (2.76 ms) is slightly slower than ResNet18 (2.38 ms), because depthwise convolutions use the GPU poorly and the latency at this size is dominated by kernel launches. At batch size 32 the three models are close (0.18, 0.19 and 0.23 ms per image for DSCNet, ResNet18 and SE-ResNet18). SE-ResNet18 doubles the batch-1 GPU latency (5.28 ms) without any accuracy gain. The lower cost of DSCNet is therefore most relevant for CPU-only deployment.

The end-to-end latency, including reading a full-resolution JPEG (about 800×665 pixels), resizing and normalization, is 33.0–35.9 ms per image for all models. Pre-processing alone takes 30.6 ms, so the model is a small share of the pipeline and the real-time bottleneck is image decoding rather than the network. These measurements were taken on a Colab T4 GPU and its CPU; they were not repeated on an edge device.

Table 8: Computational cost and latency (Tesla T4 GPU and Colab CPU).

Metric	***ResNet18***	***SE-ResNet18***	***DSCNet***
Parameters (M)	11.19	11.28	2.70
GMACs per image	0.93	0.93	0.24
GPU latency, batch 1 (ms/image)	2.38	5.28	2.76
GPU latency, batch 8 (ms/image)	0.31	0.75	0.35
GPU latency, batch 32 (ms/image)	0.19	0.23	0.18
GPU p95 latency, batch 1 (ms)	3.37	7.51	3.05
CPU latency, batch 1 (ms/image)	29.31	29.67	11.17
Throughput, batch 32 (images/s)	5,176	4,424	5,708
End-to-end, batch 1 (ms/image)	33.03	35.92	33.41

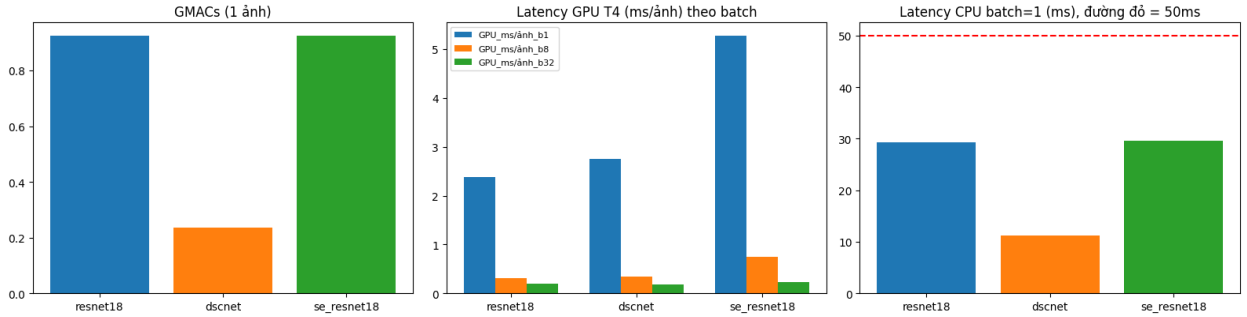


Figure 17: GMACs, GPU latency versus batch size, and CPU latency (batch 1) of the three model

4.8. Calibration and selective prediction

Reliable confidence scores matter for a diagnostic tool. The expected calibration error (ECE) [13] of the three models before temperature scaling is 0.09–0.11 (Table 9, Figure 18). For DSCNet the mean confidence (0.433) is about 6 points above its accuracy (0.373), a mild over-confidence. Temperature scaling, fitted on the validation set, gave temperatures close to 1 (0.95–1.06) and changed the ECE only slightly: 0.107 → 0.087 (ResNet18), 0.091 → 0.073 (SE-ResNet18) and 0.090 → 0.091 (DSCNet). It did not reduce the test negative log-likelihood meaningfully (DSCNet 2.153 → 2.140). The models are thus already reasonably calibrated, and the remaining error is caused by the difference between validation and test data, not by the logit scale. DSCNet is not the best-calibrated model after scaling (SE-ResNet18 has the lowest ECE) [16].

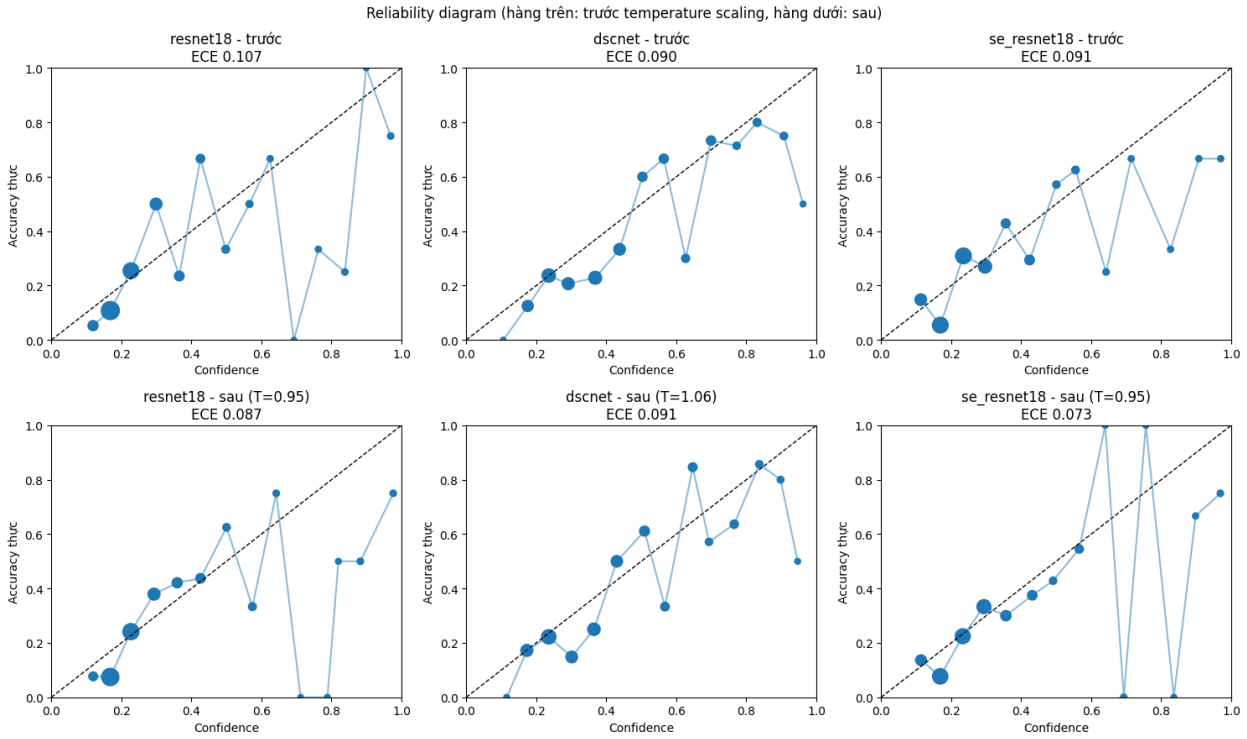


Figure 18: Reliability diagrams of the three models before and after temperature scaling

The confidence can nevertheless be used to reject uncertain images (Figure 19). With a threshold of 0.4 on the calibrated DSCNet confidence, the system answers 42.8% of the test images with an accuracy of 60.4% on the answered images (versus 37.3% without rejection); a threshold of 0.5 answers 28.8% with 64.7% accuracy, and 0.6 answers 19.1% with 73.3%. Above 0.8 fewer than 6% of images remain and the estimates become unreliable. The system can therefore return "uncertain, please retake the photo" for low-confidence inputs, trading coverage for accuracy.

Table 9: Calibration of the three models on the test set (temperature fitted on the validation set).

Model	*Calibration*	*T*	*ECE*	*NLL*	*Mean confidence*	*Accuracy*
ResNet18	before	1.000	0.107	2.348	0.287	0.267
ResNet18	after	0.946	0.087	2.349	0.302	0.267
SE-ResNet18	before	1.000	0.091	2.405	0.293	0.263
SE-ResNet18	after	0.951	0.073	2.410	0.307	0.263
DSCNet	before	1.000	0.090	2.153	0.433	0.373
DSCNet	after	1.062	0.091	2.140	0.410	0.373

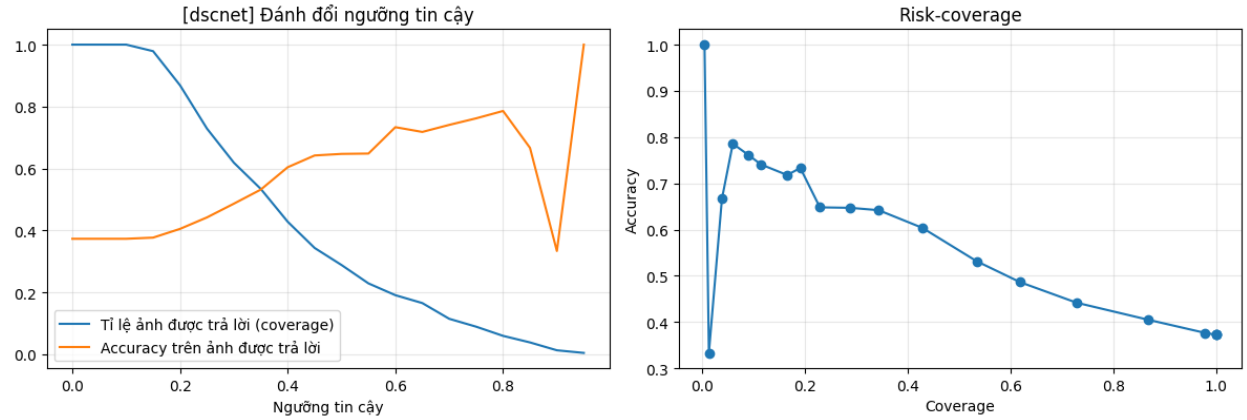


Figure 19: Trade-off between confidence threshold, coverage and accuracy of DSCNet (calibrated)

4.9. Reference experiment with ImageNet pre-training

To quantify how much of the remaining error is due to the small dataset, ResNet18 initialized with ImageNet weights [12] was fine-tuned with the same split and augmentation (learning rate 5×10^{-4}, 25 epochs, three seeds). This model is a reference only and not part of the from-scratch comparison. It reached a test accuracy of 57.9% ± 0.9%, a Top-3 accuracy of 80.1% ± 1.5% and a macro-F1 of 0.567 ± 0.015 (validation macro-F1 0.582 ± 0.009). This is 20.6 points of accuracy above DSCNet trained from scratch; the McNemar test confirms the difference (pre-trained correct / DSCNet wrong: 63 images, the reverse: 15 images; $p < 0.0001$). DSCNet from scratch therefore reaches about 64% of the accuracy of the pre-trained model. The low absolute accuracy in this

study is mostly explained by the amount of data, and transfer learning is the most promising way to improve it.

Table 10: From-scratch models versus the pre-trained reference (mean ± std over three seeds).

Model	***Val macro-F1***	***Test Acc***	***Test Top-3***	***Test macro-F1***
ResNet18 (scratch)	0.288 ± 0.011	0.258 ± 0.015	0.525 ± 0.026	0.249 ± 0.019
SE-ResNet18 (scratch)	0.285 ± 0.014	0.266 ± 0.017	0.538 ± 0.026	0.252 ± 0.013
DSCNet (scratch)	0.372 ± 0.011	0.373 ± 0.021	0.621 ± 0.011	0.354 ± 0.016
ResNet18 (ImageNet, reference)	0.582 ± 0.009	0.579 ± 0.009	0.801 ± 0.015	0.567 ± 0.015

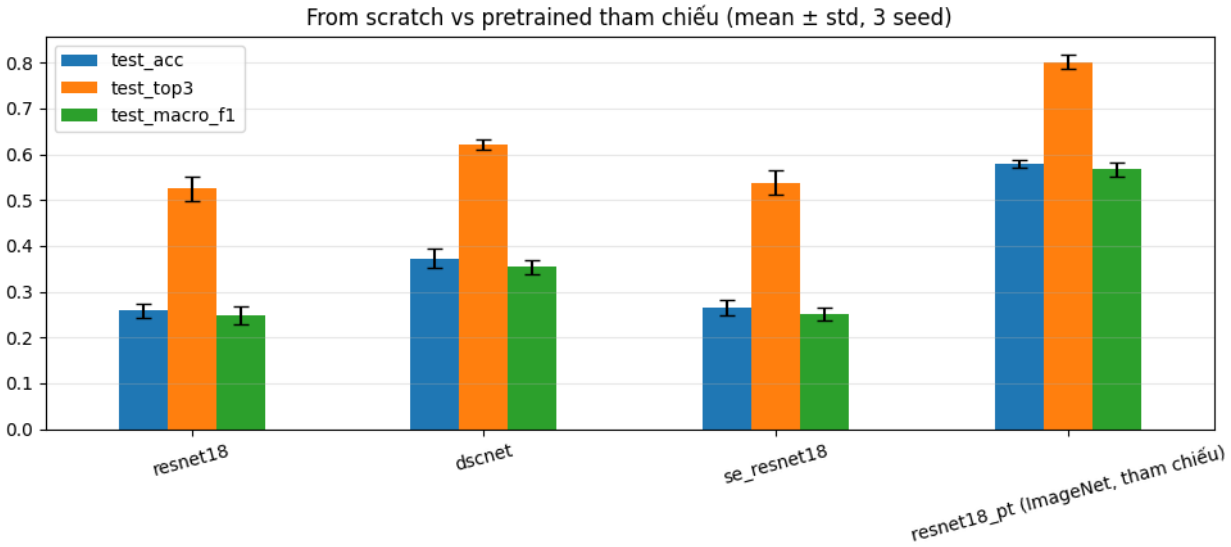


Figure 20: From-scratch models versus the ImageNet-pretrained ResNet18 reference

CHAPTER 5: ERROR ANALYSIS AND INTERPRETABILITY

DSCNet (seed 42) misclassified 148 of the 236 test images. This chapter examines why, whether the training labels are reliable, and whether the model relies on the leaf rather than on the background. The Grad-CAM analysis itself is presented in Section 3.4.

5.1. Main Sources of Classification Errors

Three observations characterize the errors of DSCNet. First, 75% of the wrong predictions are assigned to a different plant species, not to another disease of the same crop; species recognition (53.1%) is a weaker point than disease recognition within a species (70.2%). Second, confidence is informative but not decisive: the mean confidence is 0.54 for correct and 0.37 for wrong predictions, and some errors still receive high confidence (Figure 21, up to 0.96).



Figure 21: The 16 test images misclassified by DSCNet with the highest confidence (true label and predicted label with confidence)

Third, the most confident errors fall into recognizable groups: (i) corn diseases that look alike (gray leaf spot, leaf blight, rust), (ii) healthy leaves photographed as isolated stock images on uniform backgrounds with watermarks (for example a bell pepper leaf on a black background predicted as grape leaf with 0.96 confidence, and blueberry and cherry leaves predicted as apple or peach leaves), and (iii) tomato and potato diseases with similar lesions (late blight, bacterial spot, septoria, mosaic versus yellow-leaf virus). Together with the confusion pairs in Section 4.5, the dominant error mechanism is therefore the visual similarity between classes combined with the small number of training images per class, which is consistent with the low performance on the rarest classes. The 160×160 input resolution may also remove small lesion details (septoria, bacterial spot), but this was not tested in this study.

5.2. Suspect training labels

Annotation quality is a known weakness of field datasets. The 100 training images with the highest loss under DSCNet were inspected; they concentrate in a few classes (Tomato septoria leaf spot 10, Tomato leaf bacterial spot 10, Tomato mold leaf 10, Potato early blight 9, Apple scab 7, Raspberry leaf 6, followed by Blueberry, Peach, Tomato early blight and Tomato late blight).

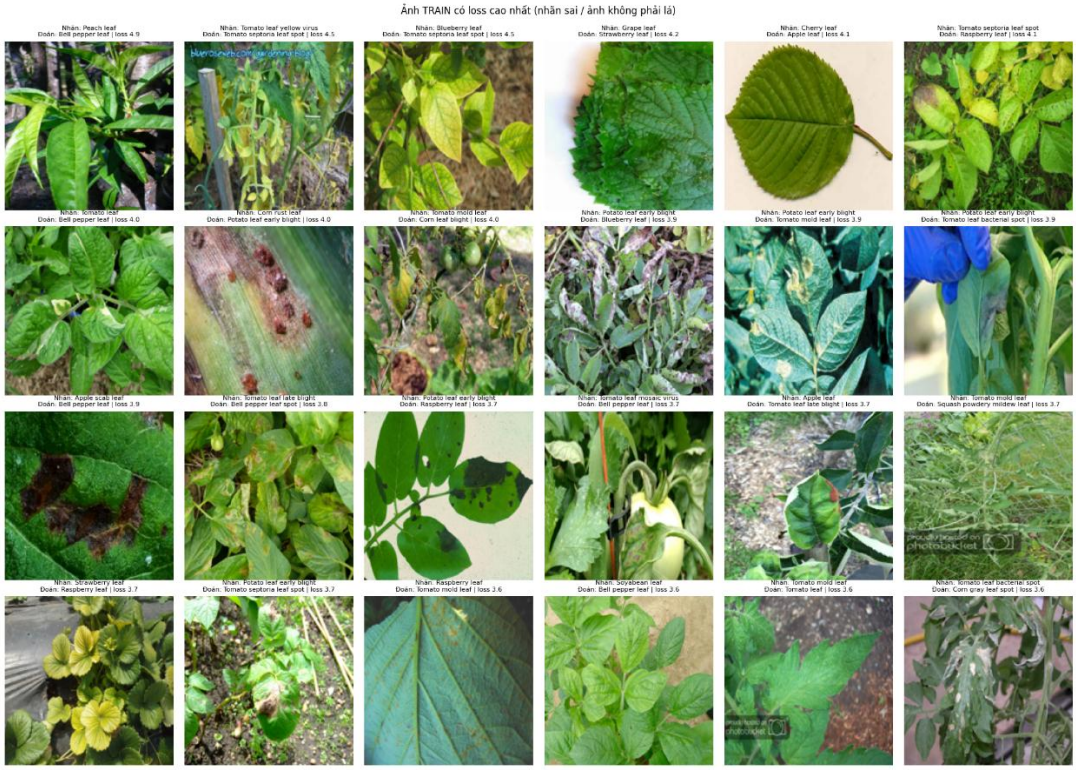


Figure 22: Training images with the highest loss under DSCNet, with their given label and the model prediction

Tomato mold and the tomato spot diseases (Figure 22) are exactly the classes with the lowest F1 in Section 4.5. High loss does not prove a wrong label, since it can also come from an atypical or ambiguous image (see [18] for loss-based detection of label errors), and the suspect images were not verified by a plant pathologist. The result suggests, however, that part of the low performance on these classes may come from noisy or ambiguous annotations and not only from the model.

5.3. Background shortcut analysis

Models trained on field images may exploit the background instead of the lesion [16]. This was tested on the 155 test images with a valid leaf mask (GrabCut-based masks, Section 3.4; chance level 0.037, margin of error about ±0.07). Four versions of each image were classified: original, blurred background, background removed (grey), and leaf removed (only the background remains). Each deletion was compared with a control that masks a random region of the same area. Without the leaf, accuracy falls to 0.05–0.08 for all models, close to chance, and the predictions collapse into 9–12 classes with the most frequent class taking 35–41% of the images. The background alone therefore carries almost no class information. Removing the leaf lowers DSCNet accuracy by 0.310, much more than the control (0.172), whereas removing the background lowers it by only 0.103, less than the control (0.206). The same pattern holds for SE-ResNet18 (leaf 0.155 vs control 0.110; background −0.013 vs 0.065). For ResNet18 the background removal (0.110) equals its control (0.105) while the leaf removal (0.194) exceeds it (0.108). In all three models the leaf region matters more than a random region of equal size, and the background matters no more than a random region. This gives no evidence of a background shortcut, although the 155-image subset and the mask quality (85% of 40 inspected masks accepted) limit the strength of this conclusion.

Table 11: Background shortcut test on 155 test images (accuracy and accuracy drop).

Model	***Original***	***Background removed***	***Leaf removed (background only)***	***Drop: background removed (control)***	***Drop: leaf removed (control)***
ResNet18	0.245	0.135	0.052	0.110 (0.105)	0.194 (0.108)
SE-ResNet18	0.226	0.239	0.071	−0.013 (0.065)	0.155 (0.110)
DSCNet	0.394	0.290	0.084	0.103 (0.206)	0.310 (0.172)

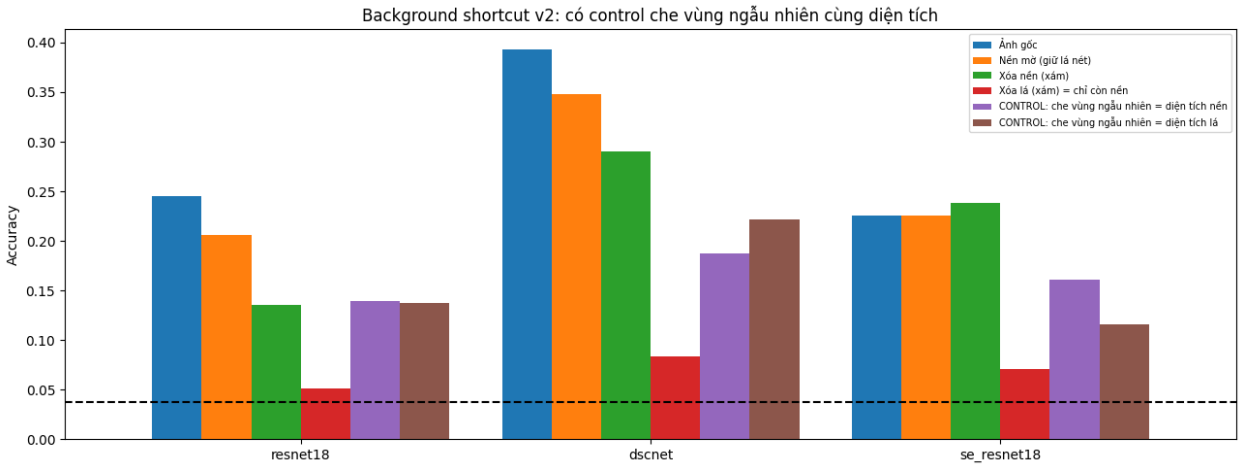


Figure 23: Accuracy under background blurring, background removal and leaf removal, compared with random-region controls (155 images)

5.4. Summary of the Grad-CAM evidence

The visual and quantitative Grad-CAM results (Figures 3–5) support only a moderate conclusion. At the final 5×5 layer, the leaf enrichment of the real Grad-CAM (1.12–1.25) is lower than that of a Gaussian map centred on the image (1.455), and it cannot be distinguished from the Grad-CAM of other images for any model; at this resolution the maps mainly reflect that leaves are usually centred. At the intermediate 10×10 layer, the maps of DSCNet and SE-ResNet18 are more specific to each image, and masking the 20% most activated area lowers accuracy more than masking a region of the same shape at a random place, for all three models (Δ = +0.153, +0.106 and +0.080 for DSCNet, SE-ResNet18 and ResNet18). The random-location control is, however, not fully neutral: it often moves the mask towards the image border, where there is less information than at the centre, so part of the effect may reflect the central position of the leaf. Grad-CAM is therefore used in this report as a qualitative explanation tool with limited quantitative support, not as proof that the model localizes disease lesions.

5.5. Example predictions

Figure 24 shows test predictions of the calibrated DSCNet with the top-3 classes. Predictions with a confidence below 0.4 are flagged as uncertain and the user is asked to retake the photo; with this threshold the system answers 43% of the test images with 60.4% accuracy (Section 4.8). Some correct predictions have high confidence (for example a corn leaf blight image with 66%), while ambiguous images are rejected, which is the intended behaviour for a screening tool.

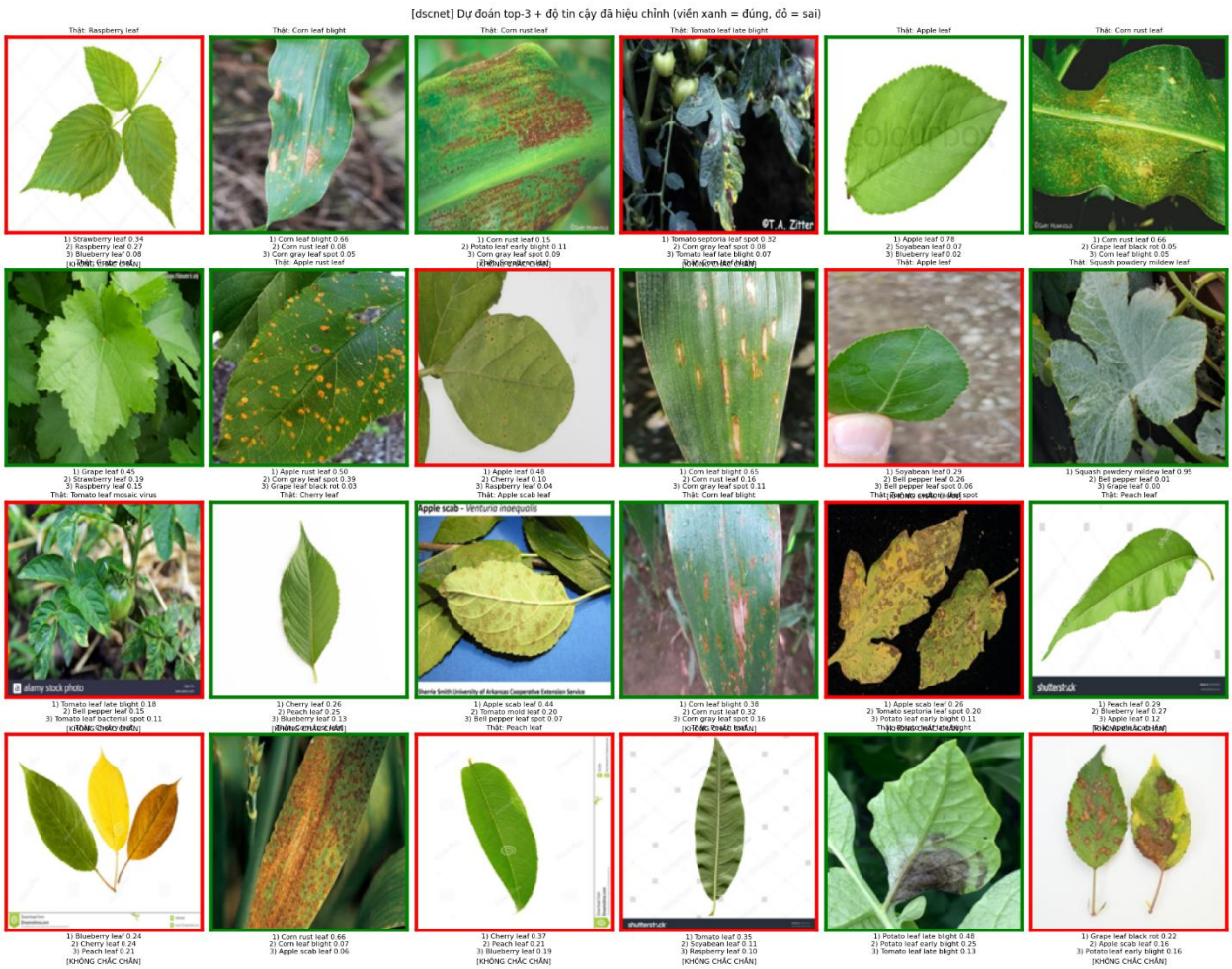


Figure 24: Example test predictions of DSCNet with top-3 probabilities and the uncertainty flag (threshold 0.4)

CHAPTER 6: CONCLUSION AND FUTURE WORK

7.1. Results

This study compared three convolutional networks trained from scratch on the cleaned PlantDoc dataset (27 classes, 1,944 training, 344 validation and 236 test images): ResNet18, SE-ResNet18 and the custom depthwise-separable DSCNet. DSCNet was the best model over three seeds, with a test accuracy of 37.3% ± 2.1%, a Top-3 accuracy of 62.1% ± 1.1% and a macro-F1 of 0.354 ± 0.016, against about 26% accuracy for the two ResNet-based models; the difference is statistically significant (McNemar $p = 0.0005$). It uses only 2.70M parameters and 0.24 GMACs, and its CPU latency is 11.2 ms per image (2.6× faster than ResNet18); the end-to-end latency of about 33 ms is dominated by image pre-processing. DSCNet separates healthy from diseased leaves with 89.4% ± 1.7% accuracy (majority baseline 61.9%), and identifies the plant species with 53.1% accuracy. Strong data augmentation was essential (test macro-F1 falls from 0.354 to 0.144 without it), whereas class weighting and batch size had no clear effect.

The experiments also showed that strong data augmentation was important for training stability and generalization. Without augmentation, the gap between training and validation performance increased substantially, indicating stronger overfitting.

7.2. Evaluation

The results demonstrate that model size alone does not determine classification performance. Although ResNet18 and SE-ResNet18 have substantially more parameters than DSCNet, they did not achieve better accuracy under the same training-from-scratch setting.

DSCNet provided the best balance between classification performance and computational efficiency. Its smaller architecture reduces the number of parameters and computational cost, making it more suitable for resource-constrained applications.

However, the overall classification accuracy remains relatively low. This is mainly related to the limited size and severe class imbalance of the PlantDoc dataset, as well as the visual similarity between several disease categories. A pretrained ResNet18 reference achieved 57.9% accuracy, showing that transfer learning can provide a substantial advantage over training from scratch.

7.3. Discussion

The findings suggest that lightweight CNNs can be useful for plant disease screening, but their effectiveness depends strongly on the characteristics of the dataset and the training strategy.

DSCNet performed particularly well in terms of the trade-off between accuracy and computational cost. The Grad-CAM analysis in Section 7 also indicates that the model generally focuses on disease-related regions of the leaves, suggesting that its predictions are based on meaningful visual features rather than only background information.

Nevertheless, the performance gap between the scratch-trained models and the pretrained reference highlights the importance of transfer learning for small and imbalanced agricultural datasets. Therefore, future work should focus on pretrained backbones, improved class balancing, higher-resolution inputs, and validation on real-world field images.

Overall, DSCNet provides a promising lightweight baseline for plant disease classification, but further improvements are required before it can be considered reliable for practical deployment.

7.4. Future work

While DSCNet exhibits promising computational efficiency and practical viability, several directions remain for future improvement:

- Transfer Learning Integration: Investigating domain-specific self-supervised pre-training or fine-tuning from large-scale pre-trained backbones (e.g., ImageNet) to bridge the absolute accuracy gap caused by limited training sample sizes.
- Fine-Grained Attention Mechanisms: Incorporating advanced spatial-attention or multi-scale feature fusion modules to better distinguish visually overlapping disease symptoms (such as early versus late blight).
- Hardware-Specific Profiling and Deployment: Porting and benchmarking the optimized DSCNet weights onto physical edge computing devices (such as Raspberry Pi 5 or dedicated embedded hardware) to evaluate real-world power consumption, memory footprint, and thermal stability.

REFERENCES

[1] Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S., and Batra, N., 2020, "PlantDoc: A Dataset for Visual Plant Disease Detection", Proceedings of the 7th ACM IKDD CoDS and 25th COMAD, pp. 249-253. DOI: 10.1145/3371158.3371196. [Conference Proceeding]

[2] Mohanty, S.P., Hughes, D.P., and Salathé, M., 2016, "Using Deep Learning for Image-Based Plant Disease Detection", Frontiers in Plant Science, Vol. 7, Article 1419. DOI: 10.3389/fpls.2016.01419. [Scientific Journal]

[3] Hughes, D.P., and Salathé, M., 2015, "An Open Access Repository of Images on Plant Health to Enable the Development of Mobile Disease Diagnostics", arXiv:1511.08060. DOI: 10.48550/arXiv.1511.08060. [Preprint]

[4] He, K., Zhang, X., Ren, S., and Sun, J., 2016, "Deep Residual Learning for Image Recognition", Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770-778. DOI: 10.1109/CVPR.2016.90. [Conference Proceeding]

[5] Ioffe, S., and Szegedy, C., 2015, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift", Proceedings of the 32nd International Conference on Machine Learning (ICML), PMLR Vol. 37, pp. 448-456. DOI: 10.48550/arXiv.1502.03167. [Conference Proceeding]

[6] Hu, J., Shen, L., and Sun, G., 2018, "Squeeze-and-Excitation Networks", Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7132-7141. DOI: 10.1109/CVPR.2018.00745. [Conference Proceeding]

[7] Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., and Adam, H., 2017, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications", arXiv:1704.04861. DOI: 10.48550/arXiv.1704.04861. [Preprint]

[8] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C., 2018, "MobileNetV2: Inverted Residuals and Linear Bottlenecks", Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4510-4520. DOI: 10.1109/CVPR.2018.00474. [Conference Proceeding]

[9] Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D., 2017, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization", Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 618-626. DOI: 10.1109/ICCV.2017.74. [Conference Proceeding]

[10] Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., and Wichmann, F.A., 2020, “Shortcut Learning in Deep Neural Networks”, Nature Machine Intelligence, Vol. 2, No. 11, pp. 665-673. DOI: 10.1038/s42256-020-00257-z. [Scientific Journal]

[11] Rother, C., Kolmogorov, V., and Blake, A., 2004, “‘GrabCut’: Interactive Foreground Extraction Using Iterated Graph Cuts”, ACM Transactions on Graphics, Vol. 23, No. 3, pp. 309-314. DOI: 10.1145/1015706.1015720. [Scientific Journal]

[12] Loshchilov, I., and Hutter, F., 2019, “Decoupled Weight Decay Regularization”, International Conference on Learning Representations (ICLR). DOI: 10.48550/arXiv.1711.05101. [Conference Proceeding]

[13] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., and Wojna, Z., 2016, “Rethinking the Inception Architecture for Computer Vision”, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2818-2826. DOI: 10.1109/CVPR.2016.308. [Conference Proceeding]

[14] Efron, B., and Tibshirani, R.J., 1993, “An Introduction to the Bootstrap”, Chapman & Hall, New York. ISBN 0-412-04231-2. DOI: 10.1201/9780429246593. [Book]

[15] McNemar, Q., 1947, “Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages”, Psychometrika, Vol. 12, No. 2, pp. 153-157. DOI: 10.1007/BF02295996. [Scientific Journal]

[16] Guo, C., Pleiss, G., Sun, Y., and Weinberger, K.Q., 2017, “On Calibration of Modern Neural Networks”, Proceedings of the 34th International Conference on Machine Learning (ICML), PMLR Vol. 70, pp. 1321-1330. DOI: 10.48550/arXiv.1706.04599. [Conference Proceeding]

[17] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L., 2009, “ImageNet: A Large-Scale Hierarchical Image Database”, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 248-255. DOI: 10.1109/CVPR.2009.5206848. [Conference Proceeding]

[18] Northcutt, C.G., Jiang, L., and Chuang, I.L., 2021, “Confident Learning: Estimating Uncertainty in Dataset Labels”, Journal of Artificial Intelligence Research, Vol. 70, pp. 1373-1411. DOI: 10.1613/jair.1.12125. [Scientific Journal]